Star Destroyer 1/2700

Fibre-optic lighting - Arduino Nano ESP32 build guide

Print this guide and tick each box as you go. Work one stage at a time and test before moving on.

What you are building

A small **Arduino Nano ESP32** (45 x 18 mm, Bluetooth built in) controls 19 lighting channels through three **TBD62083APG driver chips**: 7 engine channels with soft start, breathing and independent shimmer, plus 12 zones. Each zone is a cluster of LEDs feeding one bundle of fibre optics: 10 hull zones of 4 LEDs, a bridge zone of 2 LEDs and a hangar bay zone of 4 LEDs. That is 53 LEDs in total. You switch everything on and off from a phone with Adafruit's free **Bluefruit Connect** app - no custom app, and no separate Bluetooth module. **Power requirement: a single 5 V USB-C charger** (any phone charger capable of USB-C Power Delivery is fine), feeding a USB-C PD breakout board in the base. No mains wiring, battery pack or other power supply is needed - the breakout board is what turns a standard charger into the clean 5 V rail the electronics run from. When you press ON the lights come on in this order:

When	What happens
Press 1 (ON)	Main engine 1 spools up (3 s smooth fade)
2.5 s / 5 s	Main engines 2 and 3 spool up
8 s	All four secondary engines spool up together
12.5 s	Bridge zone lights
16.5 s	Hangar bay zone (underside) lights
Then	One hull zone at a time lights in random order, every 10-30 s (average 20 s), until all 10 zones are on. Engines keep breathing and shimmering.
After 2 hours	Auto-off: the lights switch off by themselves (change or disable in the sketch).
Press 2 (OFF)	Engines fade out over 1.5 s, all zones switch off. Press 1 again to replay the show.

Stages

Stage	Job	Page
-	Parts list	2
1	Test fit the two controller boards in the hull (do this first)	14
2	Set up the Arduino software and test the Nano ESP32	15
3	Make the 19 LED clusters (7 engines, 12 zones)	16
4	Build the controller boards	17
5	Test every channel with the pin tester	24
6	Upload the sketch and pair your phone	25
7	Tune the engine look	26
8	Prepare and install the fibre optics	27
9	Mount and tidy the final build	28
10	Power up the pole (USB-C PD board) and troubleshooting	29
Appendix	Main sketch (copy into the Arduino IDE)	32

Please read: the engine PWM in this version uses the ESP32 LEDC calls from the Espressif documentation, and the page after the board layout lists exactly what the manufacturers document. This design has not been proven on real hardware yet. Stage 2, Stage 5 and Stage 10 include checks to run before you rely on it.

Before you start: prove it on a breadboard first

This document covers the **permanent build** - soldering the final boards and fitting them into the hull for good. Before committing to any soldering, it is worth proving the circuit and the sketches work at all, using the separate **Testing** document set: breadboard diagrams for all three driver boards plus the Nano, built with no soldering, and the same **pin_test_nano.ino** sketch used again in Stage 5 below. Stage 5 then repeats that same check against your soldered boards, to catch any soldering mistakes rather than any design mistakes - those should already be ruled out by the breadboard stage.

Parts list

Quantities include a few spares. This is a generic parts list rather than a shopping list tied to one supplier - exact part numbers, brands and availability will vary by country, so treat the named parts (the driver chip, the fuse) as a specification to match rather than an exact order code. Not needed for this build: anything described for a different controller (an Arduino Mega version of this project, for example) - this design uses only the parts below.

Before you build

1. **Driver chip:** confirm you have the **TBD62083A** variant, not the TBD62084A - the 84A needs about 7 V to switch on and will not work from the Nano's 3.3 V logic pins.
2. **Resettable fuse:** check its holding current rating against the roughly 0.5 A this design draws, with some headroom - a fuse rated too low will trip or run hot. **Check the fuse's own datasheet figures before fitting it.**
3. **Nano power:** the Nano ESP32's VIN pin needs 6-21 V, too high for this design's 5 V bus, so it is powered through its own USB-C port instead, via a short pigtail wired to the same +5 V/GND bus as the LEDs (Stage 10). VIN is left unconnected.
4. **Bluefruit Connect app:** confirm it installs on the phone you plan to control this with before you commit to the Bluetooth approach.
5. Do the Stage 1 cardboard test fit before buying or building anything else.

A. Electronics

Got?	Item	Qty	Notes
[ ]	**Arduino Nano ESP32**, with headers fitted	1	Bluetooth built in, 3.3 V logic, USB-C.
[ ]	USB-A to USB-C cable (data)	1	For programming the Nano ESP32. Must be a data cable, not charge-only.
[ ]	**TBD62083A** 8-channel driver, 18-pin DIP	4	NOT the TBD62084A (needs 7 V). See "Before you build" above.
[ ]	18-pin DIP sockets	4	So the driver chips can be removed later if needed.
[ ]	Resettable fuse (PTC), THT, rated for at least 1 A hold current	2	Check the actual figure against the load - see "Before you build" above.
[ ]	**Universal prototype board, 5 x 7 cm** (18 x 24 plain plated holes, 2.54 mm pitch, no pre-etched tracks)	2 + spare	Board 1 (Nano + Driver C) and Board 2 (Driver A + B). Often sold in packs of 10 (for example The Pi Hut Proto PCB packs). See the two-board layout in Stage 4.
[ ]	Resistors: 220 ohm and 330 ohm, 1/4 W	10 + 56	7 engine LEDs + spares, and 46 zone LEDs + spares.
[ ]	LEDs: blue-white (8), warm white (50), neutral or cool white (6)	64	5 mm, clear, narrow angle. Confirm colour temperature per role.
[ ]	15-way female header sockets, 0.1 inch pitch	2	For the Nano; rows 15.24 mm apart.
[ ]	**JST XH 2-pin right-angle socket** (side entry, S2B-XH-A or compatible), 2.5 mm pitch	7 + spares	Board 1, one per engine, openings facing the bottom edge.
[ ]	**JST XH 2-pin PCB header, vertical** (B2B-XH-A or compatible), 2.5 mm pitch	12 + spares	Board 2, one per hull zone, bridge and hangar bay. 2.5 mm pitch fits the 2.54 mm board holes. Kits sold as "JST-XH 2.54 mm" are the same part.
[ ]	**JST XH 2-pin leads**, pre-crimped female housing with red and black wires, 150-200 mm	19 + spares	One per LED cluster, plus one for the test lead. Or crimp your own: XHP-2 housings, SXH-001T-P0.6 terminals and a crimp tool for JST XH.
[ ]	Short USB-C cable or USB-C breakout board (for the power pigtail)	1	Cut it down to a plug plus two bare wires (VBUS/red and GND/black), soldered to the +5 V bus - see Stage 10. A breakout board makes it easier to identify the right pins than a bare connector shell.
[ ]	Thin insulated wire, 28-30 AWG (wire-wrap wire)	5 m	For point-to-point wiring under the boards.

Got?	Item	Qty	Notes
[ ]	14-way (2x7) shrouded box header, 0.1 inch pitch, IDC-compatible	2	One per board, for the Board 1 - Board 2 harness. Keyed so the ribbon cable only fits one way.
[ ]	14-way IDC ribbon cable with sockets on both ends	1	10-20 cm is usually enough; buy pre-made, or crimp your own from bulk ribbon cable and two IDC sockets.
[ ]	Two-core wire, about 0.25 mm2 (24 AWG)	support length + 0.5 m	Runs up through the support tube and is soldered directly to Board 1 - no plug joint, since the model is glued shut for good.
[ ]	**USB-C Power Delivery breakout board** (fixed 5 V output)	1	**Required** - this is what turns a standard USB-C charger into the 5 V supply the electronics need. Leave any voltage-select jumpers at the default 5 V.
[ ]	USB-C cable + any USB-C charger	1 + 1	**Required** - the sole power source for the whole model. Any phone charger capable of USB-C Power Delivery is fine.
[ ]	Multimeter	1	Needed to check the board and 5 V supply.

B. Fibre optics and finishing

Got?	Item	Qty	Notes
[ ]	PMMA end-glow fibre optic, 0.5 mm, about 200 m	2 x 100 m	End-glow type, not side-glow.
[ ]	1 mm drill bit and pin vice (or hand chuck)	1	For the window holes.
[ ]	Black self-amalgamating (silicone) tape	1	Holds bundles to LEDs, no heat needed.
[ ]	Super glue (CA)	1	Fibre ends.
[ ]	Matt black paint	1	Hull touch-ups and light-proofing.
[ ]	Black electrical tape	1	Light-proofing joins.

C. Structural, tools and consumables

Got?	Item	Qty	Notes
[ ]	Masking tape, cable ties, M3 screws and standoffs	assorted	Labels and mounting.
[ ]	Jumper wire pack (Dupont), male-male	1	For the breadboard test only.
[ ]	Wire strippers	1	Skip if you have some.
[ ]	Heat-shrink assortment (1.5 mm and 3 mm)	1 pack	For solder joints only. Never heat it near the fibres (Stage 8).
[ ]	Soldering iron and solder, helping hands, hobby blade, side cutters, tweezers	-	Standard electronics bench tools.

Suggested order: get the Nano ESP32 and driver chips first, so you can do the Stage 1 test fit and Stage 2 software check with the real board. Fibre optics and other finishing supplies can wait until Stage 8.

Safety: never connect an LED without its resistor. The Nano pins are 3.3 V only, so never connect 5 V to a signal pin. Unplug power and USB when changing wiring. LEDs work one way round: long leg (+) toward the resistor, short leg (-) toward the driver. Do not look straight into a lit LED at close range.

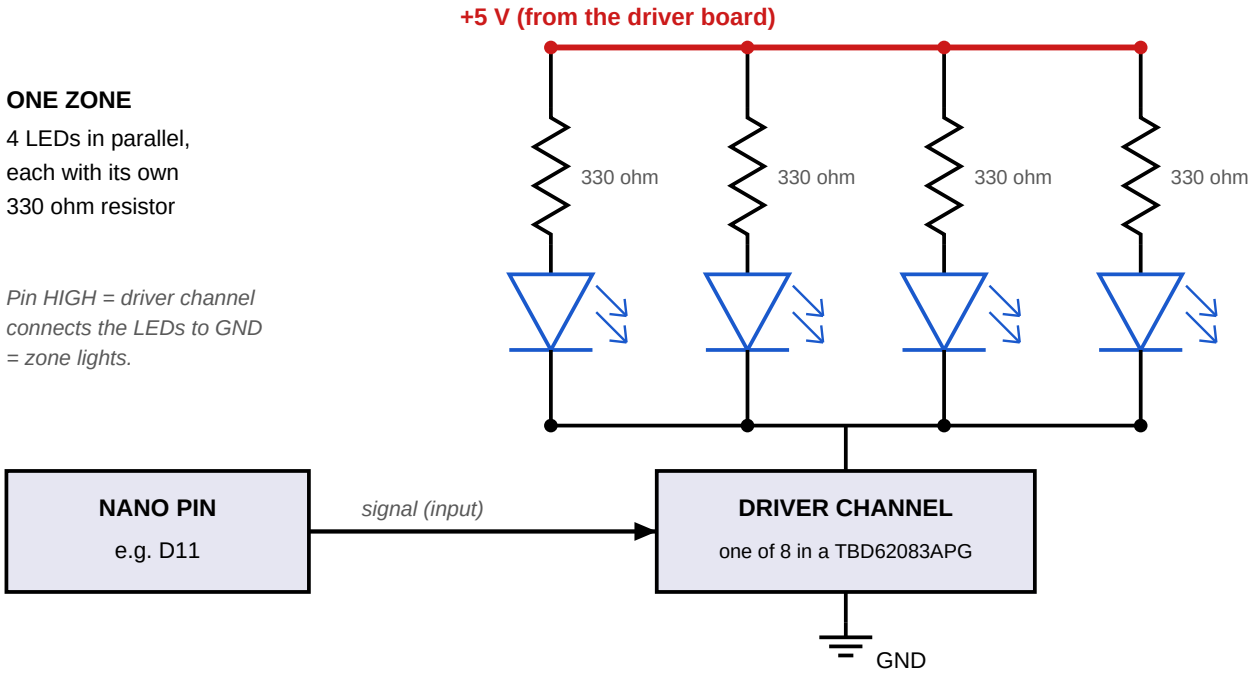
How a lighting channel works

Every channel (7 engines and 12 zones) works the same way. A cluster of LEDs is switched by one channel of a **driver chip**. The Nano ESP32 pin only sends a small 3.3 V signal to the chip, and the LEDs are powered from the 5 V supply through the chip. That is why 3.3 V logic is fine: the driver input switches on above 2.5 V. An engine is a cluster of just one LED, and the Nano dims it by pulsing the pin (PWM).

ONE ZONE

4 LEDs in parallel, each with its own 330 ohm resistor

Pin HIGH = driver channel connects the LEDs to GND = zone lights.



Reading the diagram

Each LED has its **own resistor** to +5 V: 220 ohm for an engine LED, 330 ohm for a zone LED. Never share one resistor between LEDs - they would light unevenly. All the LED cathodes (short legs) of a channel join together into one wire that goes to the driver output. When the Nano pin goes HIGH, the chip connects that wire to GND and the LEDs light. So every channel has just **two wires**: red (+5 V) and black (to the driver output).

Channel	LEDs	Colour	Resistor	Nano pin	Driver
Engines 1-7	1 each	Blue-white	220 ohm	D2 to D8	C, inputs 1-7
Hull 1 to 8	4 each	Warm white	330 ohm	D11, D12, D13, A0, A1, A3, A4, A5	A, inputs 1-8
Hull 9, Hull 10	4 each	Warm white	330 ohm	A6, A7	B, inputs 1-2
Bridge	2 (one per tower)	Warm white	330 ohm	D9	B, input 3
Hangar bay	4	Neutral / cool white	330 ohm	D10	B, input 4

Total current with everything on is about 0.4-0.5 A (estimated): roughly 46 zone LEDs x 6 mA, the engines, and the Nano ESP32 with Bluetooth running.

Not a shared string

The diagram above shows one channel's wiring in detail. It is worth being clear that the 19 channels are not linked together in one cable or daisy-chained from zone to zone. Each one is entirely independent: its own pair of wires, its own connector pins on the board, and its own driver output. The diagram below shows three channels side by side to make that clear - there is nothing connecting one zone's pair to another's.

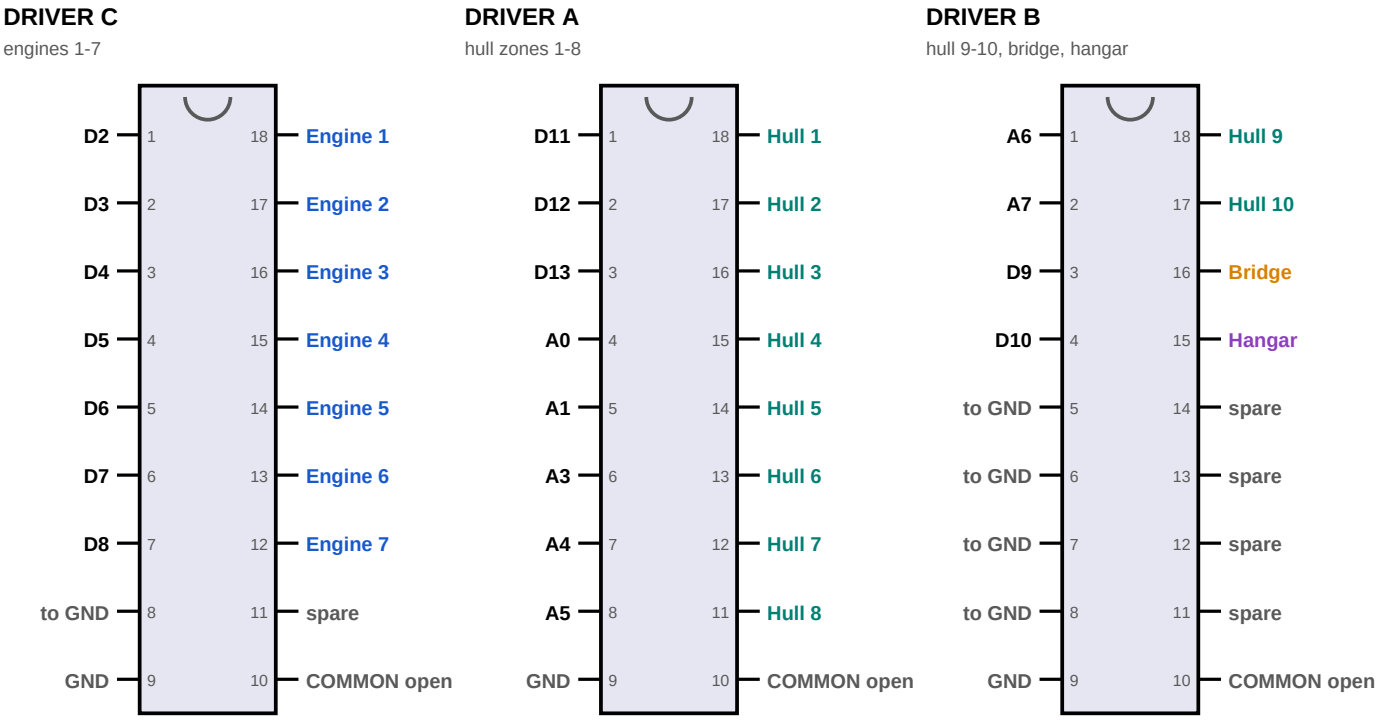
Each zone is its own independent pair - not one shared string



...16 more zones, each wired the same way, each with its own dedicated pair back to its own connector pins

Controller board wiring

Three TBD62083APG chips, split across two small boards so the electronics fit inside the hull: Driver C shares Board 1 with the Nano ESP32, while Driver A and Driver B share Board 2. Each Nano pin goes to a chip input by a short wire (or, for Board 2, a pin on the ribbon-cable harness between the boards). Each chip output goes to one 2-pin JST XH header, which the LED cluster plugs into. Use the pin numbers below - they apply the same way regardless of which board a chip sits on.



Top view of each chip, notch at the top. Left: Nano pin wired to the chip input. Right: output to a JST XH header (pin 2, OUT).

Connection	Where
Chip pin 9 (GND), all three chips	Board GND bus, which also goes to the pole GND wire and to a Nano GND pin
Chip pin 10 (COMMON), all three chips	Leave unconnected (it is only for relay and motor clamp diodes)
Driver C input 8, Driver B inputs 5 to 8	Tie to the GND bus so they cannot float and switch on
JST XH header pin 2 (OUT)	Chip output for that channel (see the diagram)
JST XH header pin 1 (+5 V)	Board +5 V bus
Pole +5 V and GND wires	Soldered directly to Board 1's bus rows (through the fuse) - no terminal block, since the model is permanent
Nano USB-C port (not VIN)	Board +5 V bus, through a soldered USB-C pigtail (VBUS+GND) - see Stage 10
Nano GND header pin	Board GND bus, same as the chips

Always use the **TBD62083A**, not the TBD62084A, which needs about 7 V to switch on.

Nano pin allocation

Use the pin names printed on the Nano ESP32. Engines use D2 to D8, which the Arduino documentation lists as PWM-capable, along with every other digital and analog pin.

Engines 1-7
driver C, inputs 1-7

D2 D3 D4 D5 D6 D7 D8

Bridge zone
driver B, input 3

D9

Hangar bay zone
driver B, input 4

D10

Hull zones 1-4
driver A, inputs 1-4

D11 D12 D13 A0

Hull zones 5-8
driver A, inputs 5-8

A1 A3 A4 A5

Hull zones 9-10
driver B, inputs 1-2

A6 A7

Do not use
leave unconnected

D0 D1 A2 VIN

Power
GND only - +5V comes in via USB-C

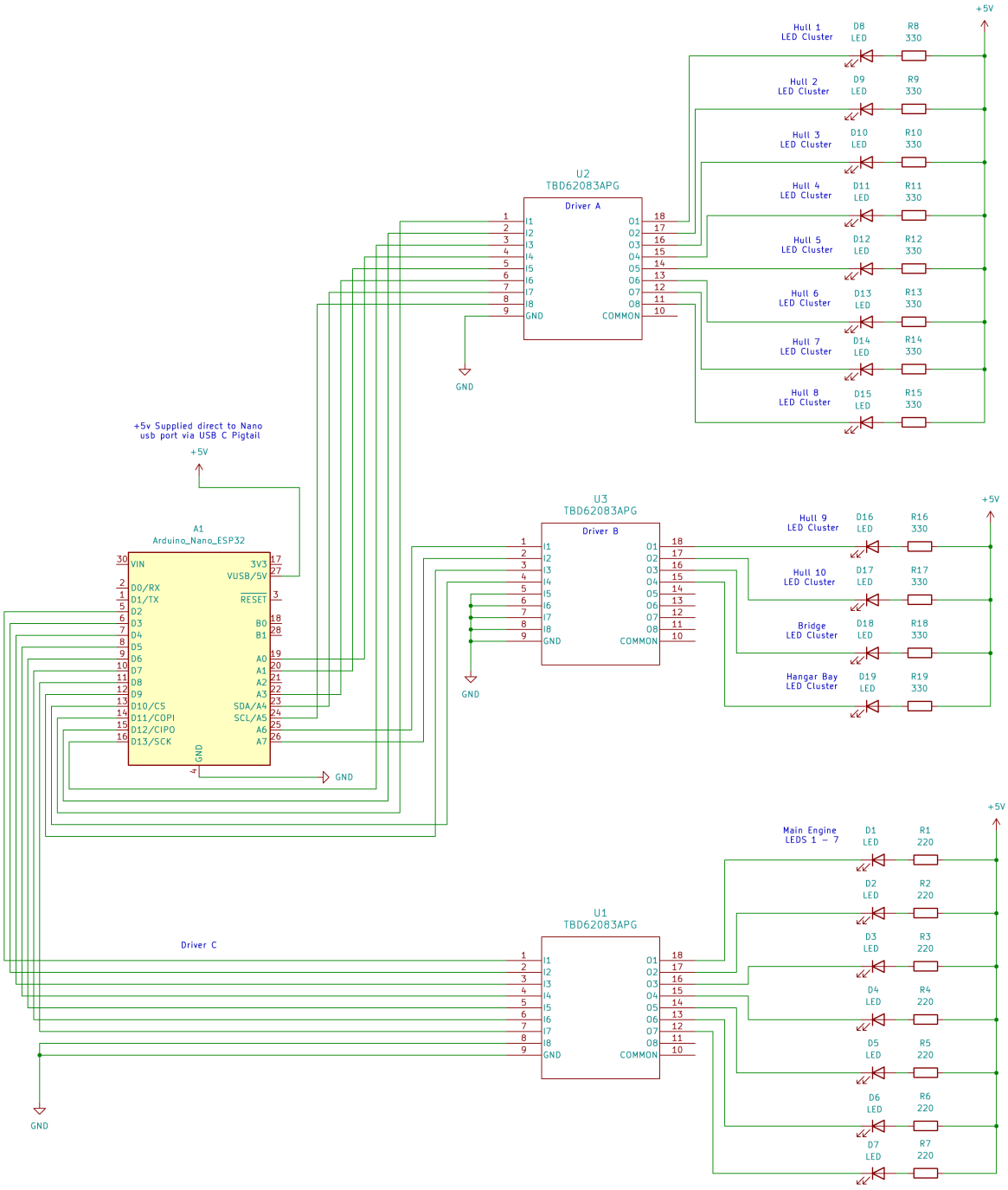
GND

Use the pin names printed on the Nano ESP32. D13 also lights the small on-board LED, which is harmless.
D0/D1 are the UART pins and A2 is a boot-strapping pin, so they are kept free. VIN is left unconnected - power comes in through the USB-C port in

Nano pin	Goes to	Channel
D2, D3, D4, D5, D6, D7, D8	Driver C inputs 1 to 7	Engines 1 to 7
D9	Driver B input 3	Bridge
D10	Driver B input 4	Hangar bay
D11, D12, D13, A0, A1, A3, A4, A5	Driver A inputs 1 to 8	Hull 1 to 8
A6, A7	Driver B inputs 1 and 2	Hull 9 and 10
VIN	Not used - left unconnected	VIN needs 6-21 V; this bus is 5 V
GND	GND bus	Common ground
USB-C port (not a header pin)	+5 V bus, via a soldered pigtail	Powers the Nano - see Stage 10
D0, D1, A2	Nothing	Kept free (UART and boot pins)

Full circuit schematic

This is the complete circuit, drawn in KiCad: the Nano ESP32, the three TBD62083APG driver chips, and the LEDs with their resistors. **To keep it clean, each LED cluster is drawn as one LED and one resistor.** In the real build every cluster holds several LEDs, each with its **own resistor**, and all the cathodes of a cluster join together onto that channel's single driver output (Stage 3 shows how one cluster is made). Hull 1 to 10 and the hangar bay have 4 LEDs each, the bridge has 2 (one per tower), and each engine has 1. Resistors are 330 ohm for zone LEDs and 220 ohm for engines.



Driver A = Hull 1-8 (inputs D11, D12, D13, A0, A1, A3, A4, A5). **Driver B** = Hull 9, Hull 10, Bridge, Hangar bay (inputs A6, A7, D9, D10; unused inputs 5-8 tied to GND). **Driver C** = Engines 1-7 (inputs D2-D8; input 8 tied to GND). Pin 10 (COMMON) is left open on every chip, A2 is left unconnected, and the Nano is powered from the +5 V supply through its USB-C pigtail (VIN is not used).

Two-board layout (Universal Proto-board PCB, 5 x 7 cm)

The electronics are split across **two boards** rather than one large one, so each piece is easy to tuck into the hull. Both are the same part: a **universal prototype board, 5 x 7 cm** - a plain grid of 18 x 24 plated holes with no pre-etched tracks, so every connection is a deliberate soldered wire.

Board 1 holds the Nano ESP32 on two sockets, Driver C (the 7-engine chip) lying on its side below it, a row of **7 JST XH headers** for the engines and the 14-way box header for the ribbon cable to Board 2. **Board 2** holds Driver A and Driver B standing upright, the matching box header and **12 JST XH headers** for the hull zones, bridge and hangar bay. Every part is drawn at its real size on the hole grid: the Nano's pin rows are 6 holes apart, each chip's pin rows 3 holes apart, and each XH header takes a patch about 3 x 3 holes.

Hole names: columns **A to X** across the top from left to right, rows **1 to 18** with **row 1 at the bottom**, seen from the component side. "K16" means column K, row 16. Channels are always named in full ("Engine 1", "Hull 3") so they never look like a hole. The underside wiring plans and wire tables are in Stage 4.

Every channel plugs in: each XH header has **pin 1 = +5 V** (red dot) and **pin 2 = OUT**, On Board 1 the 7 engine sockets are **right-angle** XH sockets with their openings facing the bottom edge, +5 V pins in columns A, D, G, J, M, P, S; on Board 2 the headers are upright, all fitted the same way round with the **lock ramp toward the bottom edge**. Make every LED lead with red on plug pin 1 and black on pin 2, and any cluster fits any channel for testing.

The harness between Board 1 and Board 2

The two boards are joined by a single **14-way IDC ribbon cable** rather than 14 loose wires. Solder a **shrouded, keyed 2x7 box header** onto each board where shown, then plug the ribbon cable's IDC sockets onto both headers. Because the header is shrouded and keyed, the cable can only go on one way round. Pin 1 (+5 V) is marked by a red stripe down one edge of the cable and a small arrow on the header - line these up at both ends.

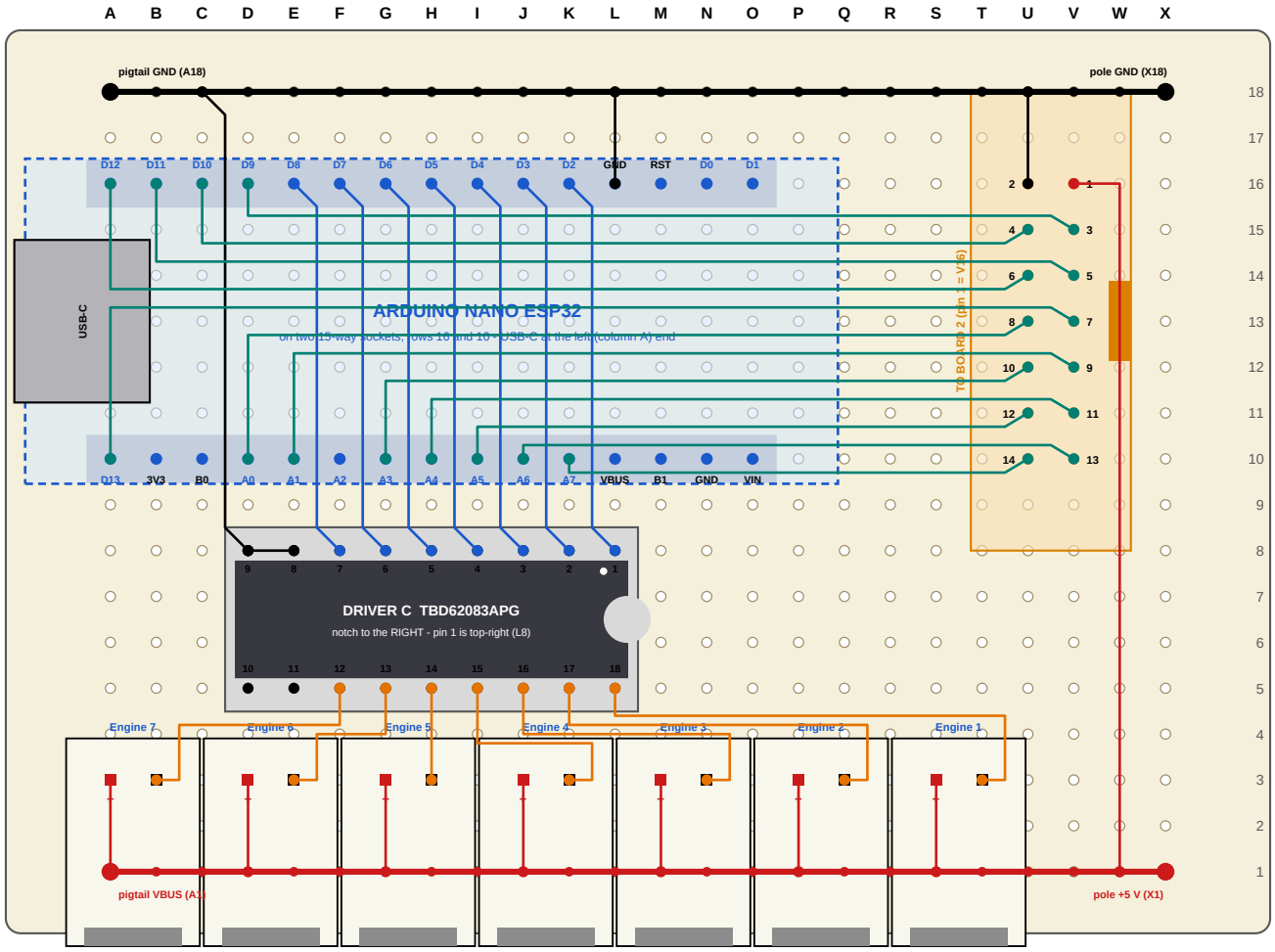
Pin	Signal	Board 1 (from)	Board 2 (to)
1	+5 V	+5 V bus	+5 V bus
2	GND	GND bus	GND bus
3	D9	Nano D9	Driver B input 3 (bridge)
4	D10	Nano D10	Driver B input 4 (hangar bay)
5	D11	Nano D11	Driver A input 1 (hull 1)
6	D12	Nano D12	Driver A input 2 (hull 2)
7	D13	Nano D13	Driver A input 3 (hull 3)
8	A0	Nano A0	Driver A input 4 (hull 4)
9	A1	Nano A1	Driver A input 5 (hull 5)
10	A3	Nano A3	Driver A input 6 (hull 6)
11	A4	Nano A4	Driver A input 7 (hull 7)
12	A5	Nano A5	Driver A input 8 (hull 8)
13	A6	Nano A6	Driver B input 1 (hull 9)
14	A7	Nano A7	Driver B input 2 (hull 10)

Power and ground overview

The pole (a support tube that also carries the two power wires) brings +5 V and ground up into the hull, where they are soldered directly onto Board 1's buses at X1 (+5 V) and X18 (GND) - no terminal block or plug joint. From there the +5 V bus feeds every engine header's pin 1, continues down the harness (pin 1) to become Board 2's +5 V bus, and also feeds the Nano ESP32 through a USB-C power pigtail plugged into its USB-C port (not VIN - see Stage 10 for why). All grounds meet the same way: Board 1's GND bus, the Nano GND pin, and harness pin 2, which becomes Board 2's GND bus. Unplug the pigtail whenever you connect the Nano to your computer by USB instead.

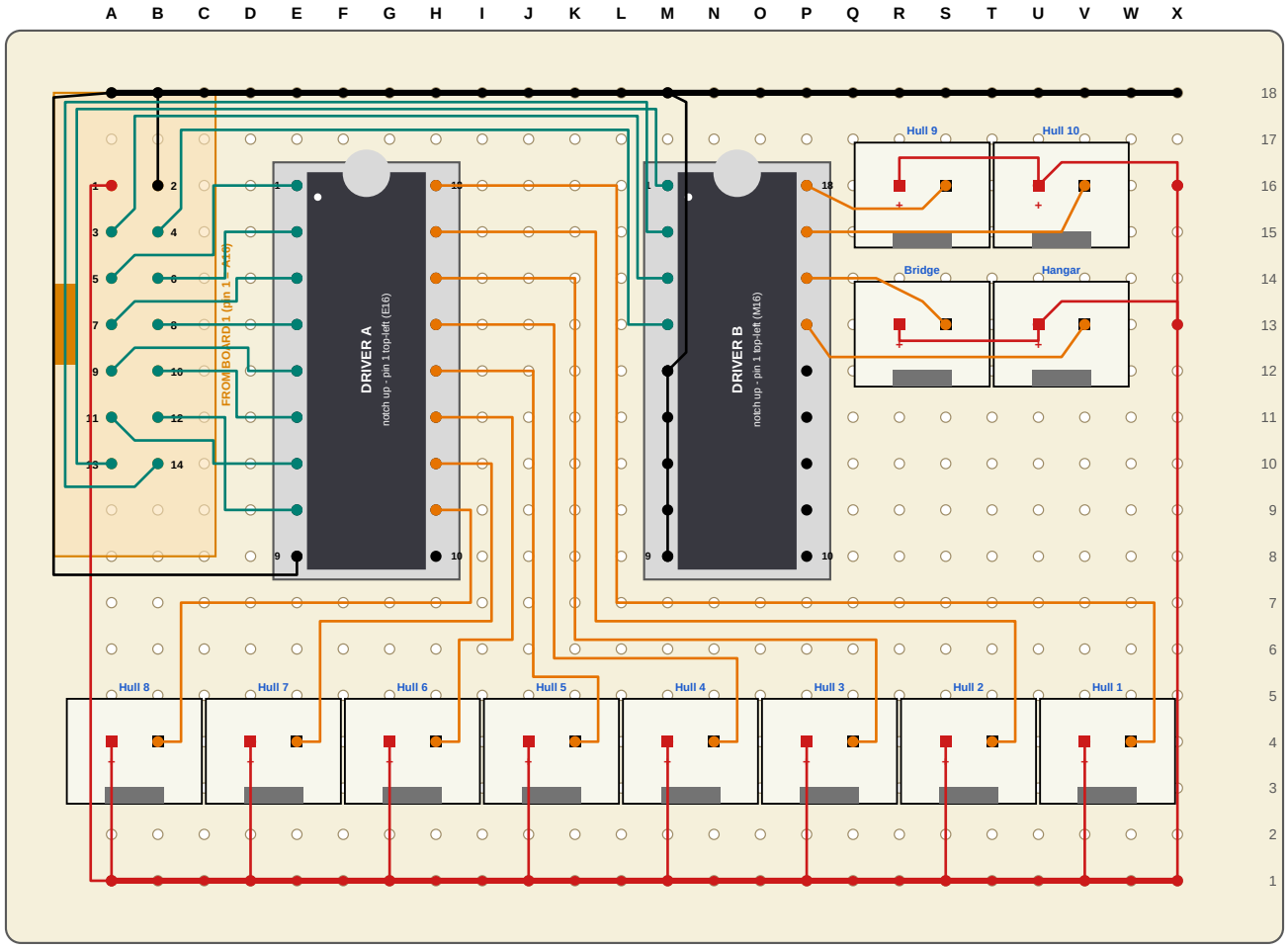
Mounting: the Nano sits about 8 mm above Board 1 in its sockets, with its USB-C port at the left edge of the board so you can reach it. Mount the two boards wherever they fit best inside the hull - they do not need to be next to each other, as long as the harness reaches. **Because the hull is glued shut for good and the power wires are soldered in permanently, there is no way to reach the electronics once the model is closed up** - not to reprogram the Nano, replace an LED, or reflow a joint. Complete every check in Stage 5 and Stage 10 first, and run the full light show from the app, before you close the hull or make the final solder joint to the pole.

Board 1 - Nano ESP32 + Driver C (component side)



Nano sockets on rows 16 and 10 (columns A-O), USB-C at the left edge. Driver C in rows 8 and 5 (columns D-L), **notch to the right, pin 1 at L8**. Engine sockets (right-angle, openings toward the bottom edge) along row 3 (Engine 7 at A-B on the left, Engine 1 at S-T on the right; +5 V pins in A, D, G, J, M, P, S). Box header at U-V, rows 16-10, pin 1 at V16. GND bus along row 18 and +5 V bus along row 1; the pole wires solder to X18 (GND) and X1 (+5 V), and the Nano's USB-C power pigtail to A18 (GND) and A1 (VBUS).

Board 2 - Driver A + Driver B (component side)



Driver A in columns E-H and Driver B in columns M-P, rows 16-8, both **notch at the top**. Box header at A-B, rows 16-10, pin 1 at A16. Hull 1-8 headers along row 4 (Hull 8 on the left, Hull 1 on the right). Hull 9, Hull 10, Bridge and Hangar headers at R-S and U-V, rows 16 and 13. GND bus along row 18, +5 V bus along row 1.

Engine PWM: what the manufacturers document

The engine pulsing uses pulse-width modulation (PWM) on seven Nano ESP32 pins. This table separates what Arduino and Espressif document from what only your own test can show.

Item	Documented value	Source
PWM pins on the Nano ESP32	All digital pins D0-D13 and all analog pins A0-A7	Arduino Nano ESP32 user manual (docs.arduino.cc)
Chip	ESP32-S3 (in the u-blox NORA-W106 module)	Arduino Nano ESP32 product documentation
Hardware PWM channels (LEDC)	ESP32-S3: 8 channels, 1 group, 4 timers	Espressif Arduino-ESP32 LEDC documentation
Duty resolution	1 to 14 bits on the ESP32-S3	Espressif Arduino-ESP32 LEDC documentation
Independent brightness per channel	Each channel owns its duty cycle; channels with the same frequency and resolution share one timer	Espressif Arduino-ESP32 LEDC documentation
LEDC clock	Default XTAL, typically 40 MHz; 8-bit minimum about 153 Hz	Espressif Arduino-ESP32 LEDC documentation
Engine PWM settings used	7 channels, 5 kHz, 12-bit (duty 0-4095): the same frequency and resolution as the Espressif LEDC software-fade example	Espressif Arduino-ESP32 LEDC documentation
Board package	Based on the Espressif 2.x core (ESP-IDF v5.1.4)	Arduino Nano ESP32 documentation (docs.arduino.cc/nano-esp32)
PWM function names	Core 2.x: ledcSetup, ledcAttachPin, ledcWrite(channel, duty). Core 3.x: ledcAttach, ledcWrite(pin, duty)	Espressif 2.x to 3.0 migration guide and LEDC documentation

Seven engine channels fit inside the 8 available, leaving one spare. The sketch checks the installed core version and uses the matching function names automatically.

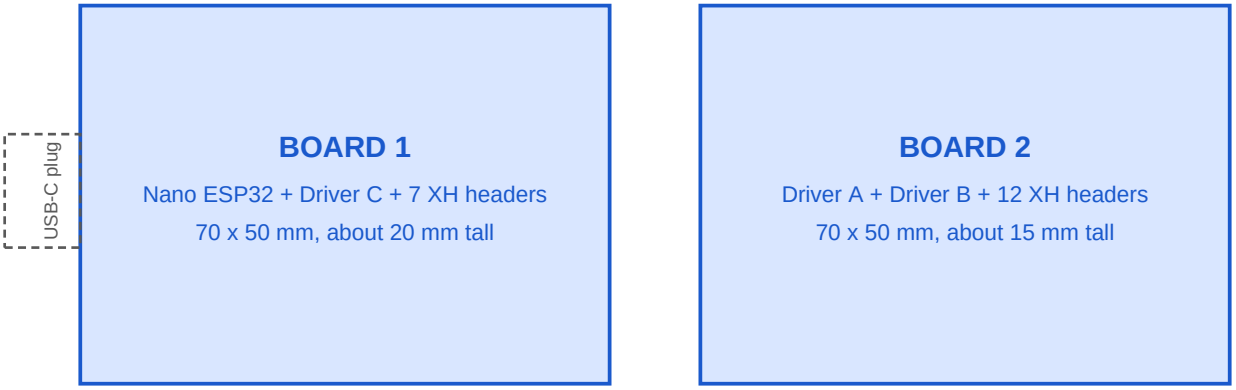
Still to confirm on your own board

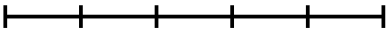
1. That all seven engines fade smoothly and independently (Stage 5, Part 2 of the pin tester).
2. That the TBD62083A driver follows 5 kHz PWM cleanly. Distributor listings for the related TBD6208x series show turn-on and turn-off delays under 1 microsecond, but check the TBD62083A datasheet yourself.
3. That the installed "Arduino ESP32 Boards" version compiles the sketch. If it reports that a ledc function is not declared, check the version shown in the Boards Manager against the engine PWM page - the sketch is written to support both core 2.x and 3.x function names.

Stage 1 - Test fit the two controller boards in the hull

Do this before you buy or build anything else. Print this page at 100 % (Actual size, no scaling) and check the scale bar measures 50 mm.

Actual size. Allow 10 mm clear above Board 1 and room above both for the XH plugs and their leads.





This bar must measure exactly 50 mm. If not, print at 100 % / Actual size.

- [ ] Cut a block of cardboard or foam to **50 x 70 mm** and build it up to **20 mm** high. This stands in for **Board 1** (Nano ESP32 + Driver C) with the Nano fitted (an estimate - the real parts are lower than this, so treat it as a worst case).
- [ ] Cut a second block, also **50 x 70 mm** but only **15 mm** high. This stands in for **Board 2** (Driver A + Driver B), which has no Nano and sits lower.
- [ ] Cut a small block, about **30 x 20 x 15 mm**, for the fuse and the pole's two power wires where they solder onto Board 1.
- [ ] Dry-fit the hull halves without glue. The two boards do not need to be next to each other - try Board 1 in one spot and Board 2 in another, wherever each fits, since a ribbon-cable harness can run between them.
- [ ] Check there is at least **10 mm clear above** Board 1's block, and about **8 mm at one end** for the USB-C plug.
- [ ] Check you can still reach the USB-C port, for example through a removable panel, so you can update the sketch later.
- [ ] Mark the best positions and see how the pole's power wires and the ribbon-cable harness will reach each board.
- [ ] Record the result: both fit [ ] one or both too tight [ ]

What to do with the result

Result	Action
Both fit	Carry on with Stage 2.
One or both too tight	Split further - a third small board, or move a single chip onto its own board - is easy with this design, since each board only needs a short harness to reach the others. Re-plan the split around whatever space is actually available, keeping each driver chip with its own short run of wires back to its XH headers.

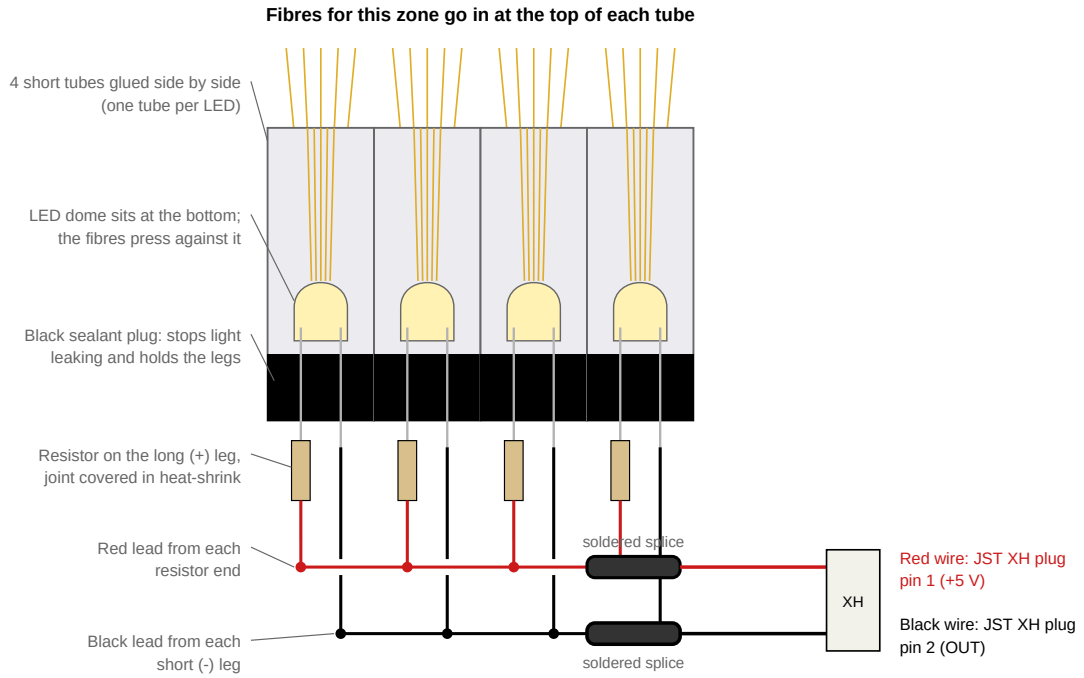
Stage 2 - Set up the Arduino software and test the Nano ESP32

- [ ] Download and install the free Arduino IDE (version 2) from arduino.cc/software.
- [ ] Open **Tools > Board > Boards Manager**, search for **Arduino ESP32 Boards** and install it.
- [ ] Plug the Nano ESP32 into your computer with the USB-C data cable.
- [ ] Choose **Tools > Board > Arduino ESP32 Boards > Arduino Nano ESP32** and the correct **Port**.
- [ ] Check **Tools > Pin Numbering** is set to **By Arduino pin (default)**. The sketches use the pin names D2, A0 and so on.
- [ ] Open **File > Examples > 01.Basics > Blink** and click Upload. The on-board LED should blink.
- [ ] Open **Tools > Board > Boards Manager** again and note the version number shown for **Arduino ESP32 Boards**. You may need it if a compile error appears. The sketches choose the correct PWM function names for version 2.x or 3.x automatically.
- [ ] If the board is not found or Upload fails, double-press the RESET button (a green LED lights) to enter bootloader mode, choose the port again and upload.

Stage 3 - Make the 19 LED clusters

Each channel is a cluster of LEDs with one +5 V wire (red) and one wire to the driver (black). Engines: 1 blue-white LED each with a **220 ohm** resistor. Hull 1 to 10: 4 warm white LEDs each. Bridge: 2 warm white. Hangar bay: 4 neutral or cool white. Zone LEDs use **330 ohm** resistors. Build one first, then repeat.

Each LED lives in its own short tube, so the fibres go in at the top and the LED sits at the bottom, sealed so no light leaks. Because the LEDs are inside tubes, you cannot twist their bare legs together. Instead, each LED becomes a small **pigtail** (resistor and two short insulated leads), and the pigtails are joined by **soldered splices** outside the tubes. Each group of red or black leads is soldered to one wire of a JST XH 2-pin lead, which plugs into the board.



Every join is soldered and covered in heat-shrink - not just twisted.

- [ ] Cut the tubes (black heat-shrink or silicone tube, about 25-30 mm long), one per LED: 4 per hull zone or the hangar bay, 2 for the bridge.
- [ ] Trim the LED legs to about 12 mm. Slide 1.5 mm heat-shrink onto each leg **before** soldering.
- [ ] Solder the right resistor to the **long leg (+)** of every LED. Slide the heat-shrink over the joint and shrink it, keeping the heat off the LED body.
- [ ] Solder a 60-80 mm **red** flying lead (thin insulated wire) to the free resistor end, and a **black** one to the **short leg (-)**. Cover both joints with heat-shrink and check nothing bare can touch.
- [ ] Push each LED into the bottom of its tube, dome facing up, with the two leads hanging out. Seal the bottom with black hot-melt glue or black silicone, working it around the legs. Once set, check that no light shows through the seal.
- [ ] Glue the tubes together side by side (one hull zone is a block of four) and label the block with its channel, for example "Engine 1", "Hull 4", "Bridge", "Hangar".
- [ ] Gather the **red** leads of the zone, strip 5 mm, twist them together with the stripped red wire of a **JST XH 2-pin lead**, then **solder** the joint and cover it with heat-shrink. Do the same with the **black** leads and the lead's black wire. Check the red wire goes to **pin 1** of the XH plug and the black to **pin 2** - make every lead the same way round. For a single-LED engine there is no splice: solder the XH lead straight on.
- [ ] **Quick test:** plug the cluster into any XH header on a powered board and touch that header's pin 2 (OUT) to GND with a wire. All the LEDs in the block should light evenly.
- [ ] Repeat for all 19 channels. The fibres are fitted into the tubes later, in Stage 8.

Stage 4 - Build the controller boards

Build Board 1 completely first and test it (Stage 5 can check Board 1 alone), then Board 2. The separate **Perfboard build in stages** sheet splits this into short sessions, each ending with its own multimeter tests; the order below is the same.

How the wiring works: these are plain perfboards with no copper strips, so every connection is a wire soldered on the **underside**. Use thin **insulated** wire for every short wire - insulated wires may cross each other. The only bare wires are the two long **bus wires** (GND along the top row, +5 V along the bottom row), and they go in **last**, once every short wire is in place. No wire ever crosses a bare bus. It will look like a small rat's nest under each board - that is normal for this kind of build.

Order of work, for each board

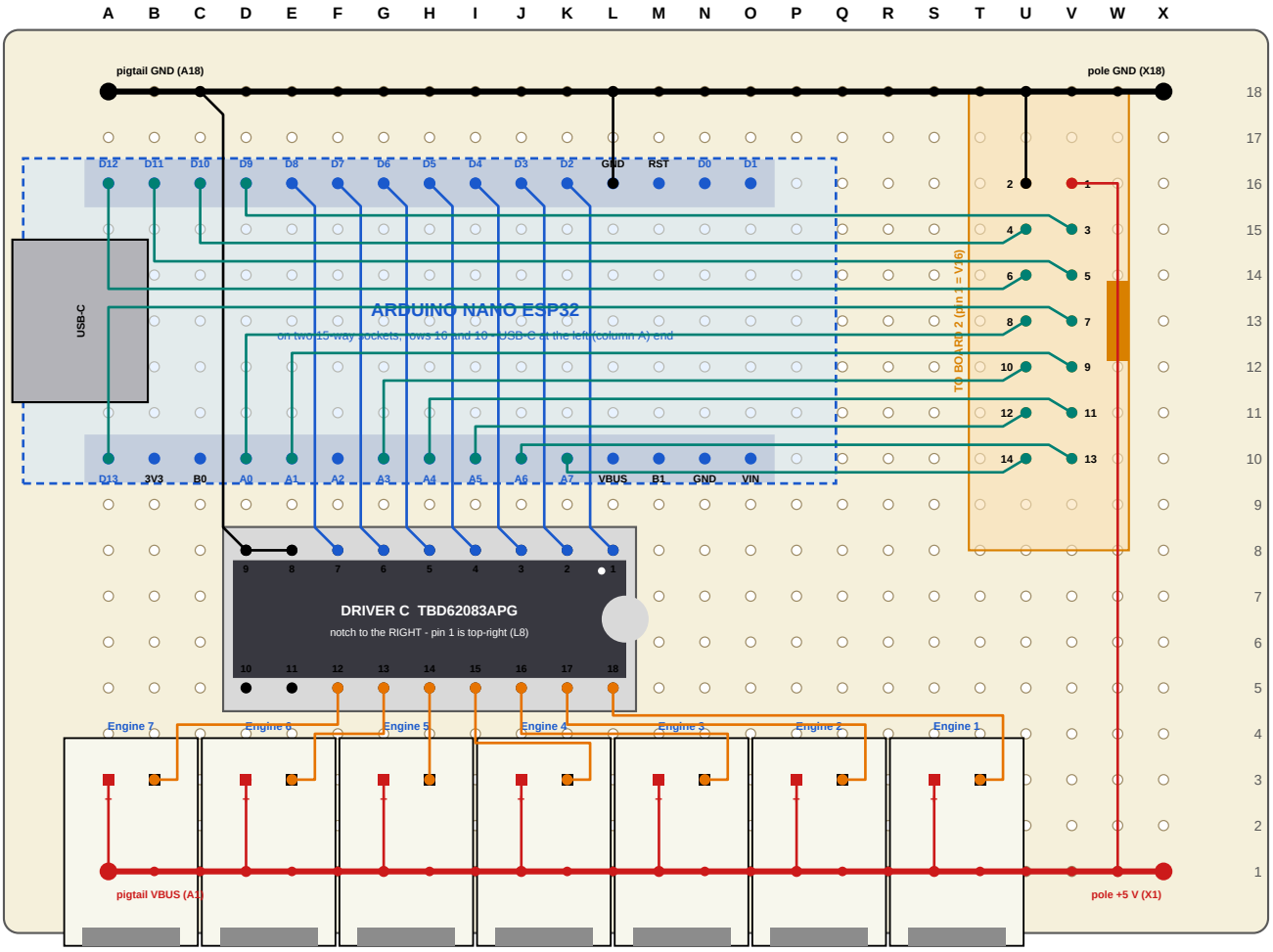
- [ ] **Sockets and headers first.** Solder the Nano sockets (Board 1), the 18-pin DIP sockets, the box header and the **JST XH headers** where the top view shows them. On Board 1 the engine sockets are right-angle, openings facing the bottom edge, +5 V pin on the left (columns A, D, G, J, M, P, S); on Board 2 fit every upright header the same way round, lock ramp toward the bottom edge, pin 1 (+5 V) on the left. Tack one pin, check the part is flat and square, then solder the rest. Leave the chips and the Nano out.
- [ ] **Short power and ground wires.** Fit the red and black wires from the table: each XH header's pin 1 down to the +5 V bus row, and the ground wires up to the GND bus row. End each one at its bus-row hole with a short bare tip.
- [ ] **Bus wires last.** Lay a straight bare tinned wire (about 0.6 mm) along row 18 (GND) and row 1 (+5 V), A to X, and solder it at every hole.
- [ ] **Check (power off):** the +5 V bus does not beep to the GND bus anywhere.
- [ ] **Signal wires.** Fit the blue and teal wires one at a time and tick each one off in the table.
- [ ] **Output wires.** Fit the orange wires from the driver outputs to each XH header's pin 2 (OUT).
- [ ] **Check (power off):** each wire beeps end to end, and no wire beeps to its neighbour.
- [ ] **Fit the chips and the Nano.** Driver C on Board 1 lies on its side with the **notch to the right**; Driver A and Driver B on Board 2 stand with the **notch at the top**. The Nano's USB-C end is at the left edge of Board 1.
- [ ] Board 1 only: make the Nano's **USB-C power pigtail** from A1 (VBUS) and A18 (GND). Do not plug it into the Nano until bench testing (Stage 5) is done. The Nano's VIN pin is **not** used.
- [ ] Connect the **ribbon cable** between the two box headers, red stripe to pin 1 at both ends.

Channel	Nano pin	Driver input pin	Driver output pin
Engine 1 to 7	D2 to D8	C pins 1 to 7	C pins 18, 17, 16, 15, 14, 13, 12
Hull 1 to 8	D11, D12, D13, A0, A1, A3, A4, A5	A pins 1 to 8	A pins 18, 17, 16, 15, 14, 13, 12, 11
Hull 9, Hull 10	A6, A7	B pins 1, 2	B pins 18, 17
Bridge, Hangar bay	D9, D10	B pins 3, 4	B pins 16, 15

Unused pins: Driver C pin 8 and Driver B pins 5-8 are inputs tied to GND so they cannot float; every chip's pin 10 (COMMON) and its spare outputs stay unconnected.

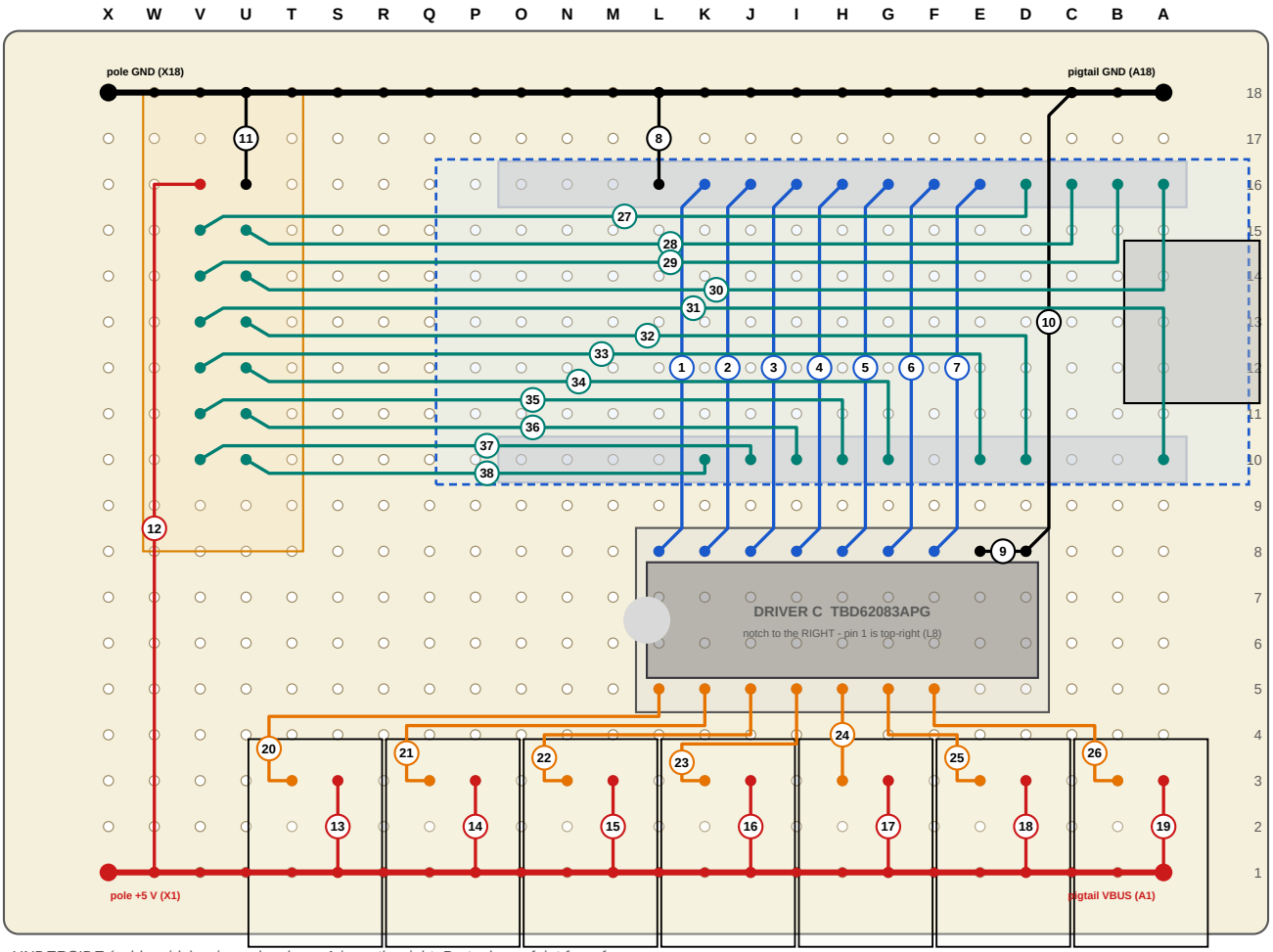
Stage 4 (continued) - Board 1: component side

Column A on the left. The underside plan on the next page shows the same board turned over.



Stage 4 (continued) - Board 1: underside

Turned over like the page of a book, so column A is on the right. Each wire carries its number from the table. A fine marker pen on the board helps you see what is done.



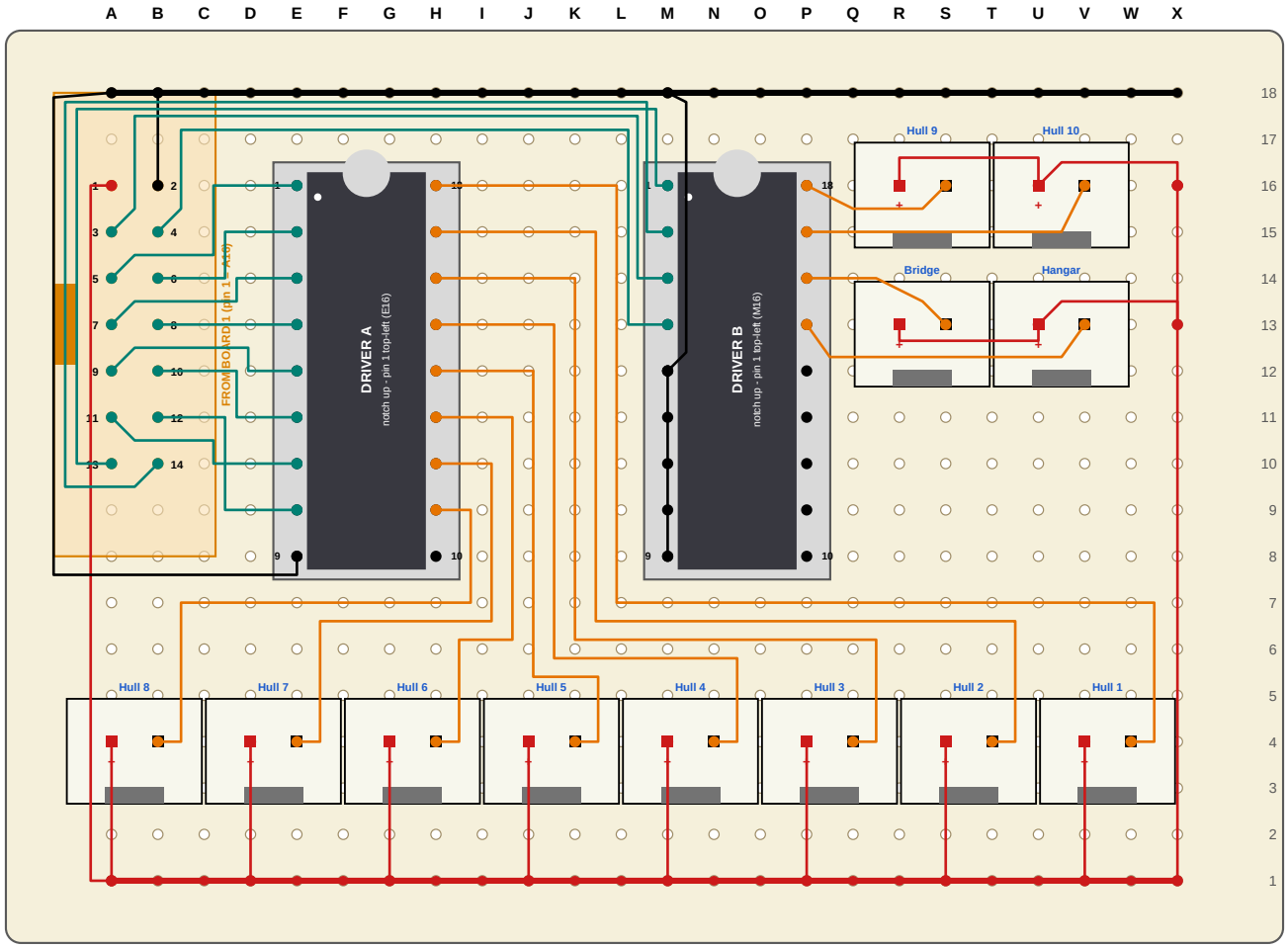
UNDERSIDE (solder side), mirrored: column A is on the right. Parts drawn faint for reference.

Board 1 - wires to fit (tick each one)

Buses (bare, laid last): GND along row 18 and +5 V along row 1, A to X. Pole wires: X1 (+5 V) and X18 (GND). Pigtail: A1 (VBUS) and A18 (GND).

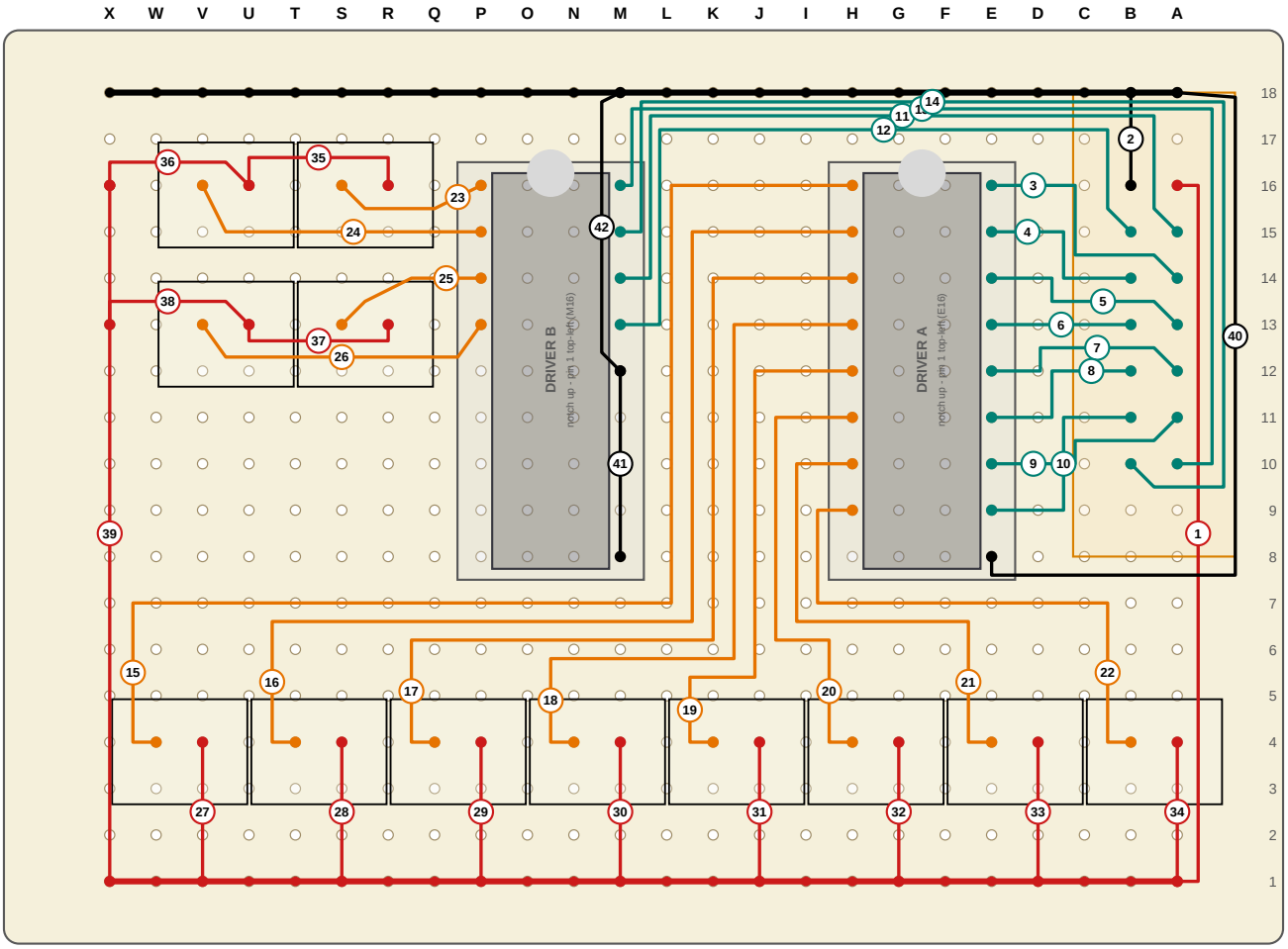
Tick	#	Colour	What it joins	From	To
[ ]	1	Blue	Nano D2 to Driver C pin 1 (engine 1)	K16	L8
[ ]	2	Blue	Nano D3 to Driver C pin 2 (engine 2)	J16	K8
[ ]	3	Blue	Nano D4 to Driver C pin 3 (engine 3)	I16	J8
[ ]	4	Blue	Nano D5 to Driver C pin 4 (engine 4)	H16	I8
[ ]	5	Blue	Nano D6 to Driver C pin 5 (engine 5)	G16	H8
[ ]	6	Blue	Nano D7 to Driver C pin 6 (engine 6)	F16	G8
[ ]	7	Blue	Nano D8 to Driver C pin 7 (engine 7)	E16	F8
[ ]	8	Black	Nano GND to the GND bus	L16	L18
[ ]	9	Black	Driver C pin 8 (unused input) to pin 9	E8	D8
[ ]	10	Black	Driver C pin 9 (GND) to the GND bus	D8	C18
[ ]	11	Black	Box header pin 2 (GND) to the GND bus	U16	U18
[ ]	12	Red	Box header pin 1 (+5 V) to the +5 V bus	V16	W1
[ ]	13	Red	Engine 1 socket pin 1 (+5 V) to the +5 V bus	S3	S1
[ ]	14	Red	Engine 2 socket pin 1 (+5 V) to the +5 V bus	P3	P1
[ ]	15	Red	Engine 3 socket pin 1 (+5 V) to the +5 V bus	M3	M1
[ ]	16	Red	Engine 4 socket pin 1 (+5 V) to the +5 V bus	J3	J1
[ ]	17	Red	Engine 5 socket pin 1 (+5 V) to the +5 V bus	G3	G1
[ ]	18	Red	Engine 6 socket pin 1 (+5 V) to the +5 V bus	D3	D1
[ ]	19	Red	Engine 7 socket pin 1 (+5 V) to the +5 V bus	A3	A1
[ ]	20	Orange	Driver C pin 18 to Engine 1 socket pin 2 (OUT)	L5	T3
[ ]	21	Orange	Driver C pin 17 to Engine 2 socket pin 2 (OUT)	K5	Q3
[ ]	22	Orange	Driver C pin 16 to Engine 3 socket pin 2 (OUT)	J5	N3
[ ]	23	Orange	Driver C pin 15 to Engine 4 socket pin 2 (OUT)	I5	K3
[ ]	24	Orange	Driver C pin 14 to Engine 5 socket pin 2 (OUT)	H5	H3
[ ]	25	Orange	Driver C pin 13 to Engine 6 socket pin 2 (OUT)	G5	E3
[ ]	26	Orange	Driver C pin 12 to Engine 7 socket pin 2 (OUT)	F5	B3
[ ]	27	Teal	Nano D9 to box header pin 3	D16	V15
[ ]	28	Teal	Nano D10 to box header pin 4	C16	U15
[ ]	29	Teal	Nano D11 to box header pin 5	B16	V14
[ ]	30	Teal	Nano D12 to box header pin 6	A16	U14
[ ]	31	Teal	Nano D13 to box header pin 7	A10	V13
[ ]	32	Teal	Nano A0 to box header pin 8	D10	U13
[ ]	33	Teal	Nano A1 to box header pin 9	E10	V12
[ ]	34	Teal	Nano A3 to box header pin 10	G10	U12
[ ]	35	Teal	Nano A4 to box header pin 11	H10	V11
[ ]	36	Teal	Nano A5 to box header pin 12	I10	U11
[ ]	37	Teal	Nano A6 to box header pin 13	J10	V10
[ ]	38	Teal	Nano A7 to box header pin 14	K10	U10

Stage 4 (continued) - Board 2: component side



Stage 4 (continued) - Board 2: underside

Mirrored in the same way (column A on the right).



UNDERSIDE (solder side), mirrored: column A is on the right. Parts drawn faint for reference.

Board 2 - wires to fit (tick each one)

Buses (bare, laid last): GND along row 18 and +5 V along row 1, A to X.

Tick	#	Colour	What it joins	From	To
[ ]	1	Red	Box header pin 1 (+5 V) to the +5 V bus, down the left edge	A16	A1
[ ]	2	Black	Box header pin 2 (GND) to the GND bus	B16	B18
[ ]	3	Teal	Box header pin 5 (D11) to Driver A pin 1 (Hull 1)	A14	E16
[ ]	4	Teal	Box header pin 6 (D12) to Driver A pin 2 (Hull 2)	B14	E15
[ ]	5	Teal	Box header pin 7 (D13) to Driver A pin 3 (Hull 3)	A13	E14
[ ]	6	Teal	Box header pin 8 (A0) to Driver A pin 4 (Hull 4)	B13	E13
[ ]	7	Teal	Box header pin 9 (A1) to Driver A pin 5 (Hull 5)	A12	E12
[ ]	8	Teal	Box header pin 10 (A3) to Driver A pin 6 (Hull 6)	B12	E11
[ ]	9	Teal	Box header pin 11 (A4) to Driver A pin 7 (Hull 7)	A11	E10
[ ]	10	Teal	Box header pin 12 (A5) to Driver A pin 8 (Hull 8)	B11	E9
[ ]	11	Teal	Box header pin 3 (D9) to Driver B pin 3 (Bridge)	A15	M14
[ ]	12	Teal	Box header pin 4 (D10) to Driver B pin 4 (Hangar)	B15	M13
[ ]	13	Teal	Box header pin 13 (A6) to Driver B pin 1 (Hull 9)	A10	M16
[ ]	14	Teal	Box header pin 14 (A7) to Driver B pin 2 (Hull 10)	B10	M15
[ ]	15	Orange	Driver A pin 18 to Hull 1 header pin 2 (OUT)	H16	W4
[ ]	16	Orange	Driver A pin 17 to Hull 2 header pin 2 (OUT)	H15	T4
[ ]	17	Orange	Driver A pin 16 to Hull 3 header pin 2 (OUT)	H14	Q4
[ ]	18	Orange	Driver A pin 15 to Hull 4 header pin 2 (OUT)	H13	N4
[ ]	19	Orange	Driver A pin 14 to Hull 5 header pin 2 (OUT)	H12	K4
[ ]	20	Orange	Driver A pin 13 to Hull 6 header pin 2 (OUT)	H11	H4
[ ]	21	Orange	Driver A pin 12 to Hull 7 header pin 2 (OUT)	H10	E4
[ ]	22	Orange	Driver A pin 11 to Hull 8 header pin 2 (OUT)	H9	B4
[ ]	23	Orange	Driver B pin 18 to Hull 9 header pin 2 (OUT)	P16	S16
[ ]	24	Orange	Driver B pin 17 to Hull 10 header pin 2 (OUT)	P15	V16
[ ]	25	Orange	Driver B pin 16 to Bridge header pin 2 (OUT)	P14	S13
[ ]	26	Orange	Driver B pin 15 to Hangar header pin 2 (OUT)	P13	V13
[ ]	27	Red	Hull 1 header pin 1 (+5 V) to the +5 V bus	V4	V1
[ ]	28	Red	Hull 2 header pin 1 (+5 V) to the +5 V bus	S4	S1
[ ]	29	Red	Hull 3 header pin 1 (+5 V) to the +5 V bus	P4	P1
[ ]	30	Red	Hull 4 header pin 1 (+5 V) to the +5 V bus	M4	M1
[ ]	31	Red	Hull 5 header pin 1 (+5 V) to the +5 V bus	J4	J1
[ ]	32	Red	Hull 6 header pin 1 (+5 V) to the +5 V bus	G4	G1
[ ]	33	Red	Hull 7 header pin 1 (+5 V) to the +5 V bus	D4	D1
[ ]	34	Red	Hull 8 header pin 1 (+5 V) to the +5 V bus	A4	A1
[ ]	35	Red	Hull 9 pin 1 to Hull 10 pin 1 (+5 V link)	R16	U16
[ ]	36	Red	Hull 10 pin 1 to the right edge	U16	X16
[ ]	37	Red	Bridge pin 1 to Hangar pin 1 (+5 V link)	R13	U13
[ ]	38	Red	Hangar pin 1 to the right edge	U13	X13
[ ]	39	Red	Right-edge +5 V wire down to the +5 V bus	X16	X1
[ ]	40	Black	Driver A pin 9 (GND) to the GND bus, up the left edge	E8	A18
[ ]	41	Black	Driver B unused inputs 5-8 joined to pin 9	M12	M8
[ ]	42	Black	Driver B pins 5-9 (GND) to the GND bus	M12	M18

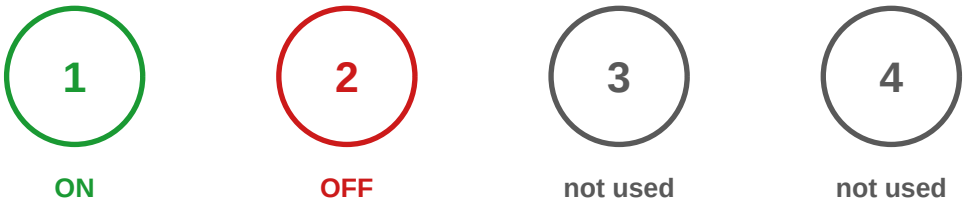
Stage 5 - Test every channel with the pin tester

This is the same check as the breadboard stage in the **Testing** document, now repeated against your newly soldered boards - it is here to catch soldering mistakes, not to re-prove the design. For this bench test the Nano is powered by your computer over USB, the normal way, and the LEDs are powered from your USB-C PD breakout board and charger - the pigtail is not involved yet. Connect the ribbon-cable harness between Board 1 and Board 2 before this test, so all 19 channels are reachable.

- **[ ]** Connect your USB-C PD breakout board (default 5 V) to Board 1's buses at X1 (+5 V) and X18 (GND) through a fuse, using temporary clip leads or a jumper wire (the pole is not soldered on permanently until Stage 9): +5 V and GND the right way round. Leave the charger unplugged for now.
- **[ ]** Plug the LED clusters' XH plugs into their headers on both boards (they only fit one way). Start with a few, then add more.
- **[ ]** Connect the Nano to your computer with the USB-C cable. Copy **pin_test_nano.ino** (from the Testing document set) into a new IDE window and upload it.
- **[ ]** Switch on the charger. Every channel should light in turn: engines 1 to 7, bridge, hangar bay, hull 1 to 10.
- **[ ]** Then the seven engines should **fade up and down one at a time**, using the Espressif LEDC calls the sketch documents. This checks that each engine pin gives its own PWM dimming. If any engine only switches on and off, or two engines fade together, stop and check your Boards Manager version against the engine PWM page before continuing.
- **[ ]** Fix any channel that does not light. Tick this only when all 19 work in the right order: [ ]

Stage 6 - Upload the sketch and pair your phone

- [ ] With the Nano still connected to your computer by USB, open a new IDE window and copy the **Appendix (main sketch)** in full.
- [ ] Click Upload. If the compile fails with an error about a Bluetooth library or an "ledc" function not declared, check your installed "Arduino ESP32 Boards" version against the engine PWM page - the sketch expects either the core 2.x or 3.x function names. The sketch is fully commented, and the companion **Main sketch explained** document walks through how the Bluetooth button protocol and the engine lighting maths work, with worked examples, if you want more detail than the comments alone give.
- [ ] After it finishes the lights stay **off** until you press ON in the app. To start the show automatically at power-up change **START_ON_BOOT** to true.
- [ ] On your phone install the free **Bluefruit Connect** app from Adafruit (iOS and Android), and allow Bluetooth access when asked.
- [ ] Open the app. A device called **StarDestroyer** should appear in the list. Tap **Connect**.
- [ ] Choose **Controller**, then **Control Pad**.
- [ ] Tap button **1**. The show should start. Tap button **2** to switch everything off, then **1** again to replay.



Control Pad buttons

Button	Action
1	ON: engines spool up in order, then the bridge, hangar bay, and hull zones at random intervals.
2	OFF: engines fade out over 1.5 s, all zones switch off.
3, 4 and the arrows	Not used by this sketch.

Notes: the lights switch themselves off after 2 hours (AUTO_OFF_MS in the sketch), and pressing 1 again restarts the show and the timer. The lights keep running if you close the app or your phone goes out of range. If the phone does not show StarDestroyer, or the buttons do nothing, see Troubleshooting.

Stage 7 - Tune the engine look

Edit the values near the top of the sketch, re-upload, and check by eye. Change one value at a time.

Setting	Default	What it does
BASE	0.70	Resting engine brightness. Raise for brighter, lower for dimmer.
BREATH_DEPTH	0.15	How much the engines swell and fade. 0 = no breathing.
BREATH_PERIOD_S	6.0	Seconds per breath. Lower is faster.
SHIMMER	0.06	Size of each LED's own random flicker. 0 = none, 0.1 = strong.
RAMP_MS	3000	Length of each soft start.
FADE_OUT_MS	1500	How long the engines take to fade out when you press OFF.
CH_START_MS	0, 2500, 5000, 8000...	When each engine starts after pressing ON.
HULL_MIN_MS / HULL_MAX_MS	10000 / 30000	Range of random gaps between hull zones.
HANGAR_ON_MS	16500	When the hangar bay lights after pressing ON (4 s after the bridge).
AUTO_OFF_MS	7200000	Auto switch-off time after pressing ON. 7200000 = 2 hours, 3600000 = 1 hour, 0 = never.
START_ON_BOOT	false	true = the show starts by itself at power-up. Handy with pole power.
PWM_FREQ / PWM_BITS	5000 / 12	Engine PWM frequency (Hz) and resolution (bits). Matches the Espressif LEDC example.

Stage 8 - Prepare and install the fibre optics

Heat warning: PMMA fibre can soften at around 100 °C (typical figure, check your fibre's datasheet), and hot air from a heat gun is far hotter than that. Use heat only on solder-joint sleeves, never near the fibres or the LED lens.

- [ ] Plan your zones: mark which windows belong to each of the 10 hull zones, the bridge, the hangar bay and the engines.
- [ ] Hangar bay: on the underside of the hull, drill the bay windows or slot with the 1 mm bit and feed the fibres in from inside. Group them into one bundle for the hangar bay zone.
- [ ] Drill each window with the 1 mm bit from the outside, going slowly. Deburr lightly.
- [ ] Push a 0.5 mm fibre through each hole from inside, leaving about 10 mm sticking out.
- [ ] Add a tiny dab of glue on the inside of each hole. Let it dry fully.
- [ ] Trim the outside fibre flush with a very sharp blade. Paint the hull; touch up over the fibre ends.
- [ ] On the inside, gather the fibres for one zone (about 15-20 per LED, so 60-80 for a 4-LED zone), tie them loosely, then split them into equal groups, one per LED. Cut the ends square.
- [ ] Each LED is already inside its own tube from Stage 3, and the four tubes of a zone are glued together. Check the tubes are clear of dust and the LED domes are at the bottom.
- [ ] Push each group of fibres into the top of its tube so the ends press against the LED dome. Secure with a dab of black glue or black self-amalgamating tape. **Do not use a heat gun or lighter near the fibres.**
- [ ] Wrap black tape over the join so no light leaks. The zone label stays on the leads.
- [ ] Power up and check each window glows evenly. Move a fibre between LEDs or add and remove fibres to balance the brightness.

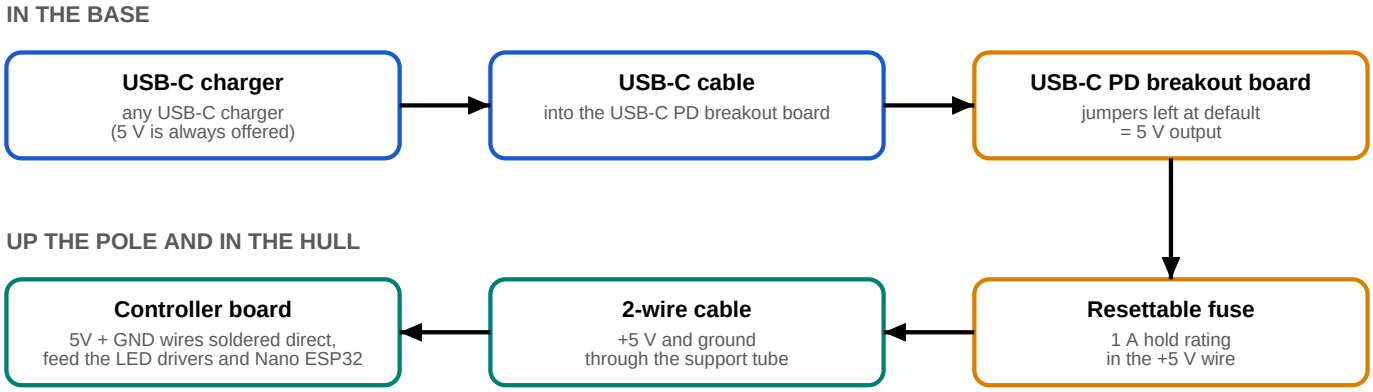
Stage 9 - Mount and tidy the final build

The JST XH headers keep every cluster pluggable, so an LED cluster can still be swapped out right up until the hull is glued. But the pole is a fixed support tube, hard-soldered to Board 1, and the hull itself is glued shut for good - so once those two things are done, **nothing inside is reachable again**, including the Nano's USB-C port unless you kept a panel over it. Board 1 and Board 2 live inside the hull, wherever they each fit best, so plan the positions you found in Stage 1.

- [ ] Mount both boards with M3 screws and standoffs, or double-sided foam, so nothing flexes.
- [ ] Keep the Nano ESP32 USB-C port reachable, for example through a removable panel - this is the only way to reprogram it once the hull is glued shut, so do not skip it. The power pigtail plugs into this same port, so make sure the panel gives enough room to unplug the pigtail and plug in a normal USB-C cable in its place.
- [ ] Plug the USB-C power pigtail you prepared in Stage 4 into the Nano's USB-C port. This becomes the Nano's only power connection once the hull is closed.
- [ ] Keep the Nano away from large metal parts so the Bluetooth signal stays strong.
- [ ] Route the ribbon-cable harness between Board 1 and Board 2, with enough slack to reach wherever you mounted them. Keep it away from sharp edges - a light crease is fine, but avoid a tight fold across the ribbon.
- [ ] Run the zone leads through the hull, grouping them by zone. Use cable ties or spiral wrap to keep them tidy.
- [ ] Feed the pole's two power wires up through the support tube and into the hull, then solder them directly onto Board 1's buses at X1 (+5 V) and X18 (GND) (see the two-board layout page). Cover each joint with heat-shrink. This is a permanent joint - there is no plug or terminal block to undo later.
- [ ] Add a dab of hot glue or a wrap of tape around each remaining plug and header pair to stop them working loose. Keep the labels visible.
- [ ] **Run the full show from the app and check every channel once more.** This is your last chance: the pole joint is now soldered and the hull is about to be glued shut, so anything you cannot fix now stays broken.
- [ ] Only once you are fully satisfied, glue the hull halves shut.

Stage 10 - Power up the pole

Only two wires (+5 V and ground) run up the support tube. The USB-C PD breakout board in the base negotiates power from a USB-C charger, with a default fixed **5 V output** - no configuration needed. The tube doubles as the model's physical support and cable conduit, and its two wires end in a permanent solder joint at Board 1 - there is no plug, socket or terminal block at the hull end.



Why the Nano is powered through its USB-C port, not VIN

The Nano ESP32's VIN pin needs **6-21 V** (confirmed in the official Arduino Nano ESP32 user manual) - this design's shared bus is 5 V, too low for VIN, so VIN is left unconnected. The Nano's USB-C port, on the other hand, is correctly rated for 5 V, so the same +5 V/GND bus that feeds the LEDs also feeds the Nano through the **USB-C power pigtail** you prepared in Stage 4 and plugged in during Stage 9 - one shared rail, no separate supply.

The pigtail only wires up VBUS and GND - the data pins are left unconnected, which is normal for a power-only connection. Most USB-C powered boards, including the Nano ESP32, power up fine from a bare VBUS+GND feed like this. If yours does not, the usual fix is to add two 5.1 kΩ resistors from the pigtail's CC1 and CC2 lines to GND, inside the USB-C plug - this is what tells a USB-C source a device is present, and some boards expect to see it even on a hard-wired feed.

Do this stage before soldering wires into the hull or gluing anything shut - test the whole power path on the bench first, while the boards are still free to move.

Power-up checklist

- [ ] On the bench, connect the USB-C PD breakout board and Board 1 with temporary clip leads (not yet through the tube) and confirm the checks below, before committing to the permanent joint.
- [ ] In the base, plug the USB-C cable into the breakout board and the other end into a USB-C charger.
- [ ] **Leave all jumpers on the board at the default 5 V setting.** Do not select a higher voltage - it would damage the LEDs and the Nano.
- [ ] Fit the two-core wire to the breakout board's output terminals. Check the +5 V and ground labels printed on your board.
- [ ] Solder a resettable fuse into the +5 V wire and cover it with heat-shrink. **Check its holding current against the load before you fit it** (see the note on the parts list) - a fuse rated well below the load can trip or run hot. The board itself does not limit current.
- [ ] Thread the two-core wire up through the support tube.
- [ ] With a multimeter, measure between +5 V and ground at the hull end while the charger is on. It should read about 5 V.
- [ ] Plug the USB-C power pigtail into the Nano and confirm it powers up (its LED lights) from this bench feed alone, with no computer USB cable attached. If it does not, see the CC resistor note above before going further.
- [ ] Now make the permanent joint: solder the pole's +5 V and GND wires directly onto Board 1's buses at X1 and X18 (Stage 9) and cover each joint with heat-shrink. This is the point of no return for the power connection.
- [ ] Switch on the charger. Open the app and press 1 to start the show. The lights switch off by themselves after 2 hours, and pressing OFF does the same - both leave the Nano powered, so unplug the charger to power down completely.
- [ ] Never power the Nano from the pole pigtail while it is also plugged into your computer by USB - unplug the pigtail first. Reprogramming is only possible through the panel left over the USB-C port (Stage 9).

Troubleshooting

Problem	Check
Nano ESP32 not found or Upload fails	Use a USB-C data cable, not the power pigtail. Double-press RESET (green LED) to enter bootloader mode, then choose the port again.
Compile error about Bluetooth or ledc	Check your installed Arduino ESP32 Boards version against the engine PWM page - the sketch expects either the core 2.x or 3.x function names, and the Bluetooth calls may need adjusting for your installed version.
StarDestroyer not in the app list	Check the sketch uploaded and the board is powered. Close and reopen the app, and allow Bluetooth access. Move the phone closer.
App connects but buttons do nothing	Use Controller then Control Pad, not another screen. Try button 1. Re-upload the sketch.
A whole channel does not light	Check the red lead is on the 5V pin and black on the OUT pin. Test the cluster alone (red on VBUS, black on GND). Check the driver chip is the right way round and its input wire goes to the right Nano pin.
Zone LEDs uneven	Each LED needs its own resistor. Check no LED is missing its resistor and all cathodes are joined.
A channel is always on	That driver input is floating or shorted. Tie unused inputs to GND and check the wiring.
No channels light	Check board +5 V and GND, chip pin 9 to GND, the Nano GND pin to the GND bus, and you used the TBD62083A (not TBD62084A).
Engines do not fade individually	Run the pin tester Part 2. If some engines only switch on and off, check the Boards Manager version against the engine PWM page.
LEDs very dim	Fibre bundle not touching the LEDs, too many fibres, or the resistors are too large.
Nothing happens on pole power	Check the fuse, +5 V and ground the right way round, the charger is on, the USB-C cable is fully in the breakout board, and the power pigtail is firmly plugged into the Nano. Measure 5 V between X1 and X18 on Board 1 before you commit to the permanent solder joint.
Nano resets or flickers on pole power, or will not power up from the pigtail at all	Supply too weak - try a different charger and a thicker wire. If it never powers up from the pigtail even with a good supply, add the two CC1/CC2 pull-down resistors described in Stage 10.
Fuse trips or runs warm	Its holding current may be below the actual load. Check its datasheet figure against your measured current and fit a higher-rated part if needed.
Light leaks into other windows	Wrap the bundle joins with black tape and paint the hull interior black.

Appendix - Main sketch (Nano ESP32 with Bluetooth)

The pin tester sketch used in Stage 5 (**pin_test_nano.ino**) is not reprinted here - it lives in the companion **Testing** document set, alongside the breadboard diagrams it was written for. This appendix has the main sketch only.

This sketch is commented throughout. For a plain-language, worked-example walkthrough of the trickier parts - the Bluetooth button protocol and the checksum that protects it, and the maths behind the engine spool-up, breathing, shimmer and gamma correction - see the companion document **Main sketch explained**, rather than relying on the comments alone.

```
// Star Destroyer lighting - ARDUINO NANO ESP32
// Control from a phone with the free Adafruit "Bluefruit Connect" app,
// using its Control Pad screen:
//   Button 1 = ON  (starts the light show from the beginning)
//   Button 2 = OFF (engines fade out, everything else switches off)
// The show also switches itself off after AUTO_OFF_MS (default 2 hours).
//
// The Nano ESP32 has Bluetooth built in (it acts like a Nordic UART device).
// All 19 lighting channels go through TBD62083APG driver chips, so a HIGH pin
// switches a channel on. Pins are 3.3 V logic.
//
// Engine PWM uses the ESP32 LEDC peripheral, documented by Espressif:
//   - Arduino ESP32 core 2.x (Arduino's Nano ESP32 package is based on 2.x):
//       ledcSetup(channel, freq, bits); ledcAttachPin(pin, channel); ledcWrite(channel, duty)
//   - Arduino ESP32 core 3.x:
//       ledcAttach(pin, freq, bits); ledcWrite(pin, duty)
// The sketch picks the right calls automatically. ESP32-S3: 8 LEDC channels,
// 1-14 bit resolution. Settings below (5 kHz, 12-bit) match Espressif's own
// LEDC software-fade example.
//
// Board package: "Arduino ESP32 Boards" > Arduino Nano ESP32
// Keep Tools > Pin Numbering on "By Arduino pin (default)".
//
// ---------------------------------------------------------------------------
// HOW TO READ THIS FILE
// The sketch is organised top to bottom as: pin and timing settings, then
// Bluetooth plumbing, then a handful of small helper functions, and finally
// the two functions every Arduino sketch has - setup() (runs once at power-up)
// and loop() (runs over and over, forever, for as long as the board is on).
// Nearly everything interesting happens inside loop() calling out to the
// helper functions above it. A companion document, "Main sketch explained",
// walks through the trickier parts (the Bluetooth packet format and the
// engine lighting maths) with worked examples, if the short comments here
// are not enough on their own.
// ---------------------------------------------------------------------------

#include <math.h>
#include <BLEDevice.h>     // Bluetooth Low Energy (BLE) support built into the ESP32 core
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>       // a small "descriptor" BLE needs so the phone can subscribe to notifications
#if __has_include("esp_arduino_version.h")
#include "esp_arduino_version.h"   // only exists on newer ("core 3.x") installs - used below to detect which one we
#endif

// Arduino's ESP32 support has changed its PWM function names between major
// versions. This checks which version is installed and remembers the answer
// in PWM_CORE_V3 (1 = new core 3.x names, 0 = older core 2.x names), so the
// rest of the sketch can just write "#if PWM_CORE_V3" wherever it needs to
// call a PWM function, instead of repeating this check everywhere.
#if defined(ESP_ARDUINO_VERSION_MAJOR) && (ESP_ARDUINO_VERSION_MAJOR >= 3)
#define PWM_CORE_V3 1
#else
#define PWM_CORE_V3 0
#endif

// ---------- PINS ----------
// Engines: 7 PWM channels (driver C, inputs 1-7)
// These are the only 7 pins that get dimmed/pulsed (see "Engine PWM" below).
// Every other lighting channel is a simple on/off digital pin.
const uint8_t N_CH = 7;
const uint8_t CH_PINS[N_CH] = {D2, D3, D4, D5, D6, D7, D8};
// Zones (driver B and driver A) - these just switch fully on or fully off,
// they never dim.
const uint8_t BRIDGE_PIN = D9;     // driver B, input 3
const uint8_t HANGAR_PIN = D10;    // driver B, input 4
const uint8_t HULL_PINS[] = {D11, D12, D13, A0, A1, A3, A4, A5, A6, A7};
const uint8_t N_HULL = sizeof(HULL_PINS);   // works out to 10 automatically from the list above

// ---------- ENGINE PWM (LEDC) ----------
// PWM (pulse-width modulation) is how a digital pin - which can only be fully
```

```
// on or fully off - is made to look dimmed: it is switched on and off very
// fast (here, 5000 times a second), and the fraction of each cycle spent on
// is the "duty cycle". A duty of 0 looks off, half the maximum looks half
// bright, and the maximum duty looks fully on. The ESP32's LEDC peripheral
// does this switching in hardware, so the sketch only has to choose a duty
// value and the chip handles the actual on/off switching by itself.
const uint32_t PWM_FREQ = 5000;                    // how many on/off cycles per second (Hz)
const uint8_t PWM_BITS = 12;                       // duty is stored as a 12-bit number...
const uint32_t PWM_MAX = (1UL << PWM_BITS) - 1;    // ...so the brightest possible duty is 2^12 - 1 = 4095

// ---------- OPTIONS ----------
const bool START_ON_BOOT = false;             // true = show starts the instant the board is powered, with no button
const unsigned long FADE_OUT_MS = 1500;       // engine fade-out time (in milliseconds) when switched off
const unsigned long AUTO_OFF_MS = 7200000UL;  // auto switch-off time in milliseconds (7200000 = 2 hours, 0 = never)

// ---------- TIMING (milliseconds after pressing ON) ----------
// Everything below is measured as "how many milliseconds after the ON
// button was pressed", not as a clock time - so CH_START_MS[1] = 2500 means
// "engine 2 starts 2.5 seconds into the show", every time the show runs.
const unsigned long CH_START_MS[N_CH] = {0, 2500, 5000, 8000, 8000, 8000, 8000};  // one start time per engine, in th
const unsigned long RAMP_MS = 3000;           // how long each engine takes to smoothly spool up once its start time
const unsigned long BRIDGE_ON_MS = 12500;     // when the bridge lights switch on
const unsigned long HANGAR_ON_MS = 16500;     // when the hangar bay lights switch on
const unsigned long HULL_MIN_MS = 10000;      // hull zones light up one at a time, waiting at least this long betwee
const unsigned long HULL_MAX_MS = 30000;      // ...and at most this long, picked randomly each time (see loop(), bel
const unsigned long UPDATE_MS = 20;           // how often the engine brightness is recalculated (50 times a second)

// ---------- ENGINE LOOK (tweak these) ----------
// These five numbers shape how the engines behave once they are running -
// see updateEngines() further down, and the companion "Main sketch
// explained" document, for exactly how each one is used in the maths.
const float BASE = 0.70;             // resting brightness once spooled up (0.0-1.0)
const float BREATH_DEPTH = 0.15;     // how far brightness rises and falls as the engines "breathe"
const float BREATH_PERIOD_S = 6.0;   // how many seconds one breath (one rise-and-fall cycle) takes
const float SHIMMER = 0.06;          // how much small, fast, random-looking flicker is layered on top
const float GAMMA = 2.2;             // reshapes the fade curve so it looks smooth and natural to the eye, not linear

// ---------- BLUETOOTH (Nordic UART service) ----------
// These three UUIDs are a published standard called the "Nordic UART
// Service" (NUS). They are not made up for this project - the Bluefruit
// Connect app specifically looks for this exact service, so that it knows
// how to talk to the board. RX is the characteristic the phone writes
// button-press data into; TX is one the board could use to send data back
// to the phone (unused here, since the lights do not need to report
// anything, but BLE requires it to be present to match the standard).
#define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define RX_UUID      "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define TX_UUID      "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

BLEServer *pServer = nullptr;
volatile bool bleConnected = false;   // "volatile" because this is changed from a BLE callback, not from loop() itse
bool wasConnected = false;
volatile uint8_t pendingCmd = 0;   // 1 = ON, 2 = OFF (set by Bluetooth, read and cleared at the top of loop())

// The show is always in exactly one of these three states:
//   IDLE    - everything off, waiting for the ON button
//   RUNNING - the full show is playing (engines spooling/breathing, zones lighting up over time)
//   FADING  - OFF was just pressed; engines are fading down before returning to IDLE
enum State { IDLE, RUNNING, FADING };
State state = IDLE;

// Per-engine "shimmer" randomness (see updateEngines()). Each engine gets
// its own pair of random flicker speeds (f1/f2) and starting points (p1/p2)
// so that the seven engines do not all flicker in exact unison.
float f1[N_CH], f2[N_CH], p1[N_CH], p2[N_CH];
float curLevel[N_CH];    // each engine's current brightness (0.0-1.0), updated every tick
float fadeFrom[N_CH];    // snapshot of curLevel[] taken the instant OFF is pressed, used as the starting point for t
uint8_t order[sizeof(HULL_PINS)];   // a shuffled list of hull-zone indexes - see shuffleHull()
uint8_t nextHull = 0;                // how many hull zones have been switched on so far this run
bool bridgeOn = false, hangarOn = false;
unsigned long t0 = 0;          // millis() value at the moment the show started (ON pressed) - everything else is tim
unsigned long lastTick = 0;    // millis() value the last time updateEngines()/updateFade() ran
unsigned long nextHullTime = 0; // millis()-since-t0 value at which the next hull zone should switch on
unsigned long fadeStart = 0;    // millis() value at the moment OFF was pressed

// Turns on the LEDC hardware PWM channel for each of the 7 engine pins.
// This only has to run once, in setup() - after this, setEngine() is all
// that is needed to change an engine's brightness.
void pwmBegin() {
  for (uint8_t i = 0; i < N_CH; i++) {
#if PWM_CORE_V3
    ledcAttach(CH_PINS[i], PWM_FREQ, PWM_BITS);   // newer core: pick the pin directly, channel number is handled int
#else
    ledcSetup(i, PWM_FREQ, PWM_BITS);             // older core: set up PWM "channel" i (0-6, one per engine)...
```

```
    ledcAttachPin(CH_PINS[i], i);                // ...then connect that channel to the actual engine pin
#endif
  }
}

// Sets one engine's brightness. i is the engine's index into CH_PINS/CH_START_MS
// (0-6), and duty is how bright (0 = off, PWM_MAX = fully on - see "ENGINE
// PWM" above for what duty means).
void setEngine(uint8_t i, uint32_t duty) {
  if (duty > PWM_MAX) duty = PWM_MAX;   // safety clamp - never ask for more than the hardware supports
#if PWM_CORE_V3
  ledcWrite(CH_PINS[i], duty);          // newer core: write is addressed by pin
#else
  ledcWrite(i, duty);                   // older core: write is addressed by the channel number set up in pwmBegin()
#endif
}

// ---------------------------------------------------------------------------
// BLUETOOTH CONTROL PAD PROTOCOL
// The Bluefruit Connect app's "Control Pad" screen sends a fixed 5-byte
// packet every time a button is pressed or released:
//     byte 0: '!'              - a fixed start marker
//     byte 1: 'B'              - says "this is a button packet"
//     byte 2: the button number as a text character, e.g. '1' for button 1
//     byte 3: '1' if the button was just pressed, '0' if just released
//     byte 4: a checksum byte, used to catch corrupted packets
// For example, pressing button 1 sends the 5 bytes: ! B 1 1 <checksum>.
// Bluetooth delivers these bytes one at a time (onWrite() below may be
// called with only part of a packet, or with several packets stuck
// together), so feedByte() below collects bytes into a small buffer and
// only acts once it has a complete, valid 5-byte packet. See the companion
// "Main sketch explained" document for a full worked example of the
// checksum calculation.
// ---------------------------------------------------------------------------
void feedByte(uint8_t c) {
  static uint8_t buf[5];
  static uint8_t n = 0;             // how many bytes of the current packet we have collected so far
  if (n == 0 && c != '!') return;    // ignore stray bytes until we see a fresh start marker
  buf[n++] = c;
  if (n == 2 && buf[1] != 'B') { n = 0; return; }   // second byte must be 'B' (a button packet) - anything else, sta
  if (n == 5) {                      // we now have a complete 5-byte packet
    n = 0;                           // reset ready for the next packet, whatever we find below
    uint8_t sum = buf[0] + buf[1] + buf[2] + buf[3];
    if ((uint8_t)~sum == buf[4] && buf[3] == '1') {   // checksum matches, AND this is a "pressed" (not "released") e
      if (buf[2] == '1') pendingCmd = 1;   // button 1 -> ON
      else if (buf[2] == '2') pendingCmd = 2;   // button 2 -> OFF
      // any other button number is simply ignored
    }
  }
}

// Required by the BLE library: called automatically when the phone connects
// or disconnects. These just update a flag; loop() checks that flag.
class ServerCb : public BLEServerCallbacks {
  void onConnect(BLEServer *s) { bleConnected = true; }
  void onDisconnect(BLEServer *s) { bleConnected = false; }
};

// Required by the BLE library: called automatically whenever the phone
// writes data to the RX characteristic. Simply hands each byte received
// to feedByte() above, in order.
class RxCb : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *c) {
    auto v = c->getValue();
    for (size_t i = 0; i < v.length(); i++) feedByte((uint8_t)v[i]);
  }
};

// A random float between lo and hi (inclusive-ish). random(0, 10000) gives a
// whole number from 0 to 9999, which is then scaled down to a 0.0-1.0
// fraction and stretched out to sit between lo and hi.
float randFloat(float lo, float hi) {
  return lo + (hi - lo) * (random(0, 10000) / 10000.0);
}

// Switches every single channel off - all 7 engines and all 12 zones - and
// resets the engines' remembered brightness to 0. Used both at power-up and
// at the end of a fade-out.
void allOff() {
  for (uint8_t i = 0; i < N_CH; i++) { setEngine(i, 0); curLevel[i] = 0; }
  digitalWrite(BRIDGE_PIN, LOW);
  digitalWrite(HANGAR_PIN, LOW);
  for (uint8_t i = 0; i < N_HULL; i++) digitalWrite(HULL_PINS[i], LOW);
}
```

```
// Fills order[] with 0, 1, 2, ... N_HULL-1 and then shuffles that list into
// a random order, using the standard "Fisher-Yates shuffle" algorithm: walk
// backwards through the list, and at each position swap it with a random
// earlier-or-equal position. The result is used by loop() to decide which
// physical hull zone lights up 1st, 2nd, 3rd and so on each time the show
// runs, so the zones do not always light up in the same predictable order.
void shuffleHull() {
  for (uint8_t i = 0; i < N_HULL; i++) order[i] = i;
  for (uint8_t i = N_HULL - 1; i > 0; i--) {
    uint8_t j = random(i + 1);
    uint8_t t = order[i]; order[i] = order[j]; order[j] = t;
  }
}

// Called once, the moment the ON button is pressed (or at boot, if
// START_ON_BOOT is true). Resets everything the show needs and switches
// into the RUNNING state - loop() takes it from here.
void startShow() {
  shuffleHull();
  nextHull = 0;
  bridgeOn = false;
  hangarOn = false;
  t0 = millis();       // "time zero" for this run - every timing check in loop() measures from here
  state = RUNNING;
}

// Called once, the moment the OFF button is pressed. Zones switch off
// immediately, but the engines are left running so updateFade() can wind
// them down smoothly over FADE_OUT_MS rather than cutting them instantly.
void stopShow() {
  digitalWrite(BRIDGE_PIN, LOW);
  digitalWrite(HANGAR_PIN, LOW);
  for (uint8_t i = 0; i < N_HULL; i++) digitalWrite(HULL_PINS[i], LOW);
  bridgeOn = false;
  hangarOn = false;
  for (uint8_t i = 0; i < N_CH; i++) fadeFrom[i] = curLevel[i];   // remember where each engine is right now...
  fadeStart = millis();                                          // ...and when the fade began...
  state = FADING;                                                // ...so updateFade() can fade from there down to 0
}

// ---------------------------------------------------------------------------
// ENGINE BRIGHTNESS MATHS
// Called roughly 50 times a second (every UPDATE_MS) while the show is
// RUNNING. For each of the 7 engines it works out a brightness between 0.0
// and 1.0 by combining four effects, then converts that into a PWM duty
// value. The four effects, in the order they are combined below:
//   1. ramp   - a smooth 0->1 spool-up over RAMP_MS once the engine's start
//               time (CH_START_MS[i]) has arrived, using "smoothstep" easing
//               (x*x*(3-2*x)) instead of a straight line, which accelerates
//               gently into and out of the motion rather than snapping on.
//   2. breath - a slow sine wave shared by all engines, so brightness rises
//               and falls together like slow breathing, by up to
//               BREATH_DEPTH each way, once every BREATH_PERIOD_S seconds.
//   3. shimmer- a faster, smaller wobble, unique to each engine (because it
//               uses that engine's own f1/f2/p1/p2 values from setup()), so
//               the engines flicker independently rather than in lockstep.
//   4. gamma  - right at the end, pow(level, GAMMA) reshapes the 0-1 level
//               before it becomes a PWM duty. This is not an "effect" like
//               the others - it corrects for the fact that human eyes don't
//               perceive brightness in a straight line against duty cycle,
//               so without it a dimmed LED looks much brighter than its
//               number would suggest. See the companion document for a
//               worked example with real numbers.
// The companion "Main sketch explained" document walks through this
// function line by line with example numbers plugged in.
// ---------------------------------------------------------------------------
void updateEngines(unsigned long now) {
  float t = now / 1000.0;   // current time in seconds since the show started, for the sine waves below
  float breath = BREATH_DEPTH * sin(TWO_PI * t / BREATH_PERIOD_S);   // shared slow breathing wave, same for every en
  for (uint8_t i = 0; i < N_CH; i++) {
    if (now < CH_START_MS[i]) { setEngine(i, 0); curLevel[i] = 0; continue; }   // this engine hasn't started yet - s
    float x = (now - CH_START_MS[i]) / (float)RAMP_MS;   // 0.0 at this engine's start time, reaching 1.0 after RAMP_
    if (x > 1.0) x = 1.0;                                // ...and staying at 1.0 (fully spooled up) after that
    float ramp = x * x * (3.0 - 2.0 * x);                // "smoothstep" - turns the straight 0->1 ramp into an ease
    float shimmer = SHIMMER * (0.6 * sin(TWO_PI * f1[i] * t + p1[i]) +
                               0.4 * sin(TWO_PI * f2[i] * t + p2[i]));   // two sine waves of different speed/phase,
    float level = ramp * (BASE + breath + shimmer);   // combine: still spooling up? scaled down by ramp. fully up? r
    if (level < 0) level = 0;    // clamp - breath/shimmer could in theory push level slightly outside 0-1
    if (level > 1) level = 1;
    curLevel[i] = level;         // remember this, in case OFF is pressed next tick and stopShow() needs a fade-from
    setEngine(i, (uint32_t)(pow(level, GAMMA) * PWM_MAX));   // gamma-correct, then scale 0.0-1.0 up to an actual PWM
  }
}
// Called roughly 50 times a second while the show is FADING (i.e. after
```

```
// OFF was pressed). x counts from 0.0 (fade just started) to 1.0 (fade
// complete) over FADE_OUT_MS, and each engine's brightness is simply its
// fadeFrom[] level scaled down by (1.0 - x) - so an engine that was dim
// when OFF was pressed reaches zero brightness at the same moment as one
// that was bright, they just both fade down proportionally together.
void updateFade(unsigned long nowAbs) {
  float x = (nowAbs - fadeStart) / (float)FADE_OUT_MS;
  if (x >= 1.0) { allOff(); state = IDLE; return; }   // fade finished - make sure everything is properly off, and go
  for (uint8_t i = 0; i < N_CH; i++) {
    float level = fadeFrom[i] * (1.0 - x);
    setEngine(i, (uint32_t)(pow(level, GAMMA) * PWM_MAX));   // same gamma correction as updateEngines(), for a consi
  }
}

// Runs once, automatically, when the board is powered up or reset.
void setup() {
  // --- Set every lighting pin to OUTPUT mode and make sure everything starts off ---
  for (uint8_t i = 0; i < N_CH; i++) pinMode(CH_PINS[i], OUTPUT);
  pwmBegin();
  pinMode(BRIDGE_PIN, OUTPUT);
  pinMode(HANGAR_PIN, OUTPUT);
  for (uint8_t i = 0; i < N_HULL; i++) pinMode(HULL_PINS[i], OUTPUT);
  allOff();

  // --- Seed the random number generator ---
  // analogRead() on a pin with nothing connected to it (A2 here) picks up
  // tiny, unpredictable electrical noise, which - combined with micros()
  // (the number of microseconds since boot, itself effectively random
  // because it depends on exactly when the board was powered up) - gives a
  // different starting point for random()/randFloat() every time the board
  // starts, so the shimmer pattern and hull order are different each run.
  randomSeed(analogRead(A2) ^ micros());  // A2 is left unconnected
  for (uint8_t i = 0; i < N_CH; i++) {
    f1[i] = randFloat(0.3, 0.9);    // each engine's two shimmer frequencies (in Hz-ish units - see updateEngines())
    f2[i] = randFloat(1.1, 2.3);
    p1[i] = randFloat(0, TWO_PI);   // ...and its two shimmer starting phases, so engines don't flicker in sync
    p2[i] = randFloat(0, TWO_PI);
  }

  // --- Bluetooth setup ---
  // Creates a BLE "server" (the Nano ESP32 itself) advertising one service
  // (the Nordic UART Service defined above) with two characteristics: RX,
  // which the phone writes button presses into, and TX, which is required
  // by the standard but not used by this sketch. Once advertising starts,
  // the board shows up in the Bluefruit Connect app's device list as
  // "StarDestroyer".
  BLEDevice::init("StarDestroyer");
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new ServerCb());
  BLEService *svc = pServer->createService(SERVICE_UUID);
  BLECharacteristic *tx = svc->createCharacteristic(TX_UUID, BLECharacteristic::PROPERTY_NOTIFY);
  tx->addDescriptor(new BLE2902());   // required so the phone can "subscribe" to TX notifications, even though we ne
  BLECharacteristic *rx = svc->createCharacteristic(
      RX_UUID, BLECharacteristic::PROPERTY_WRITE | BLECharacteristic::PROPERTY_WRITE_NR);
  rx->setCallbacks(new RxCb());       // RxCb::onWrite() fires every time the phone sends button data
  svc->start();
  BLEAdvertising *adv = pServer->getAdvertising();
  adv->addServiceUUID(SERVICE_UUID);
  adv->setScanResponse(true);
  adv->start();

  if (START_ON_BOOT) startShow();
}

// Runs over and over, forever, for as long as the board has power. Each
// pass through loop() is very fast (much less than a millisecond), so all
// the "do this every so often" logic below works by checking millis()
// (the number of milliseconds since the board started) rather than by
// pausing with delay() - that way Bluetooth and button presses are never
// kept waiting.
void loop() {
  // --- Act on the most recent button press, if any ---
  // pendingCmd is set by feedByte() (called from the Bluetooth RX
  // callback, which can happen at any moment) and is read and cleared
  // here so the actual response happens in the main loop, not inside the
  // Bluetooth callback itself.
  if (pendingCmd) {
    uint8_t cmd = pendingCmd;
    pendingCmd = 0;
    if (cmd == 1 && state == IDLE) startShow();      // ON only does anything if we're currently idle
    if (cmd == 2 && state == RUNNING) stopShow();    // OFF only does anything if the show is currently running
  }

  // --- Keep advertising after the phone disconnects ---
```

```
  // Once a phone connects, the board stops advertising itself (that's
  // normal BLE behaviour). If it doesn't restart advertising afterwards,
  // no other phone - or the same phone reconnecting - would ever find it
  // again, so this explicitly restarts advertising the moment a
  // disconnection is noticed.
  if (bleConnected && !wasConnected) wasConnected = true;
  if (!bleConnected && wasConnected) {
    delay(300);   // a short pause some BLE stacks need to settle before advertising again
    pServer->startAdvertising();
    wasConnected = false;
  }

  // --- Update engine brightness, at most every UPDATE_MS ---
  unsigned long nowAbs = millis();
  if (nowAbs - lastTick >= UPDATE_MS) {
    lastTick = nowAbs;
    if (state == RUNNING) updateEngines(nowAbs - t0);   // nowAbs - t0 = milliseconds since the show started
    else if (state == FADING) updateFade(nowAbs);
  }

  // --- Timeline: switch zones on at their scheduled moments ---
  // All the times below (BRIDGE_ON_MS etc.) are measured the same way as
  // CH_START_MS - milliseconds since the show started (t0), not a clock
  // time - so this check simply asks "has enough time passed yet?" on
  // every pass through loop(), and only acts the first time the answer
  // becomes yes (the bridgeOn/hangarOn flags make sure of that).
  if (state == RUNNING) {
    unsigned long now = millis() - t0;
    if (AUTO_OFF_MS > 0 && now >= AUTO_OFF_MS) { stopShow(); return; }   // show has been running long enough - fade
    if (!bridgeOn && now >= BRIDGE_ON_MS) {
      digitalWrite(BRIDGE_PIN, HIGH);
      bridgeOn = true;
      // The first hull zone's reveal time is only decided now, once the
      // bridge has come on - this is what makes the hull zones start
      // appearing only after the bridge, never before it.
      nextHullTime = now + random(HULL_MIN_MS, HULL_MAX_MS + 1);
    }
    if (!hangarOn && now >= HANGAR_ON_MS) {
      digitalWrite(HANGAR_PIN, HIGH);
      hangarOn = true;
    }
    // Reveal hull zones one at a time, in the shuffled order[] from
    // shuffleHull(), each one at a random gap (between HULL_MIN_MS and
    // HULL_MAX_MS) after the previous one - so all 10 eventually light up,
    // but never on a predictable schedule.
    if (bridgeOn && nextHull < N_HULL && now >= nextHullTime) {
      digitalWrite(HULL_PINS[order[nextHull]], HIGH);
      nextHull++;
      nextHullTime = now + random(HULL_MIN_MS, HULL_MAX_MS + 1);
    }
  }
}
```